RescueRight Hardware Prototyping Guide

From Zero to Working MVP

Version: 1.0
Last Updated: October 12, 2025
Status: Complete Beginner's Guide
Team: IDEATE 2025 Team Kiwi (AKSD2103)

📋 Table of Contents

🎯 Introduction - Start Here

What You're Building

You're creating a **smart choking-rescue training vest** that measures:

- **Hand position** during Heimlich maneuver (using gyroscopes)
- **Force applied** during thrusts (using Hall effect sensors)

The vest will send this data **wirelessly** to your mobile app in real-time, providing instant feedback to trainees.

Your Current Status

✅ **What you have:**

- Working React Native app with Bluetooth integration
- ESP32 DevKit V1 microcontroller
- 4x MPU9250 gyroscope sensors (for position tracking)
- 4x SEN-CUR-006 Hall effect sensors (for force measurement)
- TCA9548A I²C multiplexer (to manage multiple sensors)

❓ **What you need to learn:**

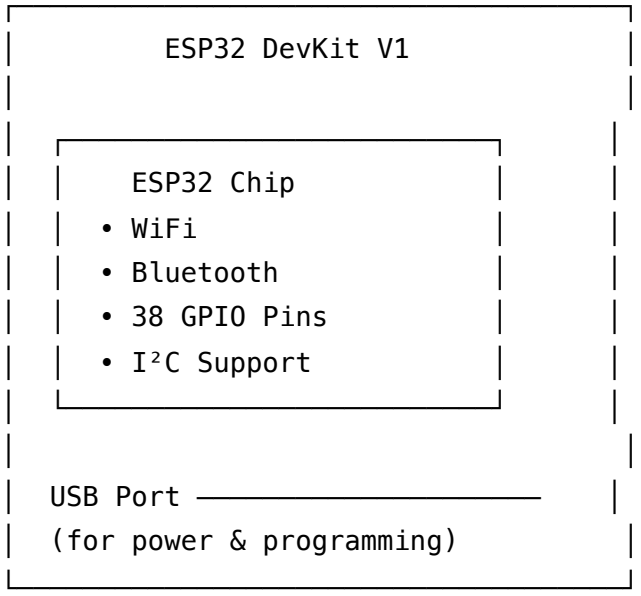
- How these components work
- How to wire them together
- How to program the ESP32
- How to test and integrate with your app

Expected Timeline

- **Hardware Assembly**: 3-4 hours
- **Software Setup**: 2-3 hours
- **Testing & Debugging**: 3-4 hours
- **App Integration**: 2-3 hours
- **Total**: 2-3 days (with breaks and learning)

🔧 Understanding Your Hardware Components

1. ESP32 DevKit V1 - Your Brain



What it does:

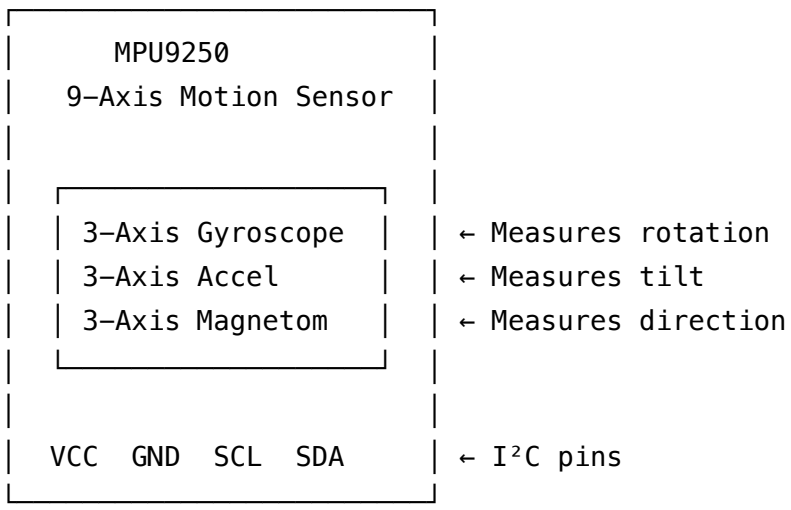
- Collects data from all sensors
- Processes the data
- Sends it wirelessly via Bluetooth to your app
- Powers everything

Key specs:

- **Voltage**: 3.3V (very important!)
- **Current**: Can provide 500mA from USB
- **GPIO Pins**: Digital input/output pins
- **I²C Pins**: Special pins for sensor communication

Think of it as: The conductor of an orchestra, coordinating all the sensors and communicating with your phone.

2. MPU9250 - Your Position Sensors



What it does:

- Measures **orientation** (which way is it tilted?)
- Measures **movement** (is it moving up, down, left, right?)
- Measures **rotation** (is it spinning?)

For your vest:
You'll place 4 of these on different parts of the torso to track:

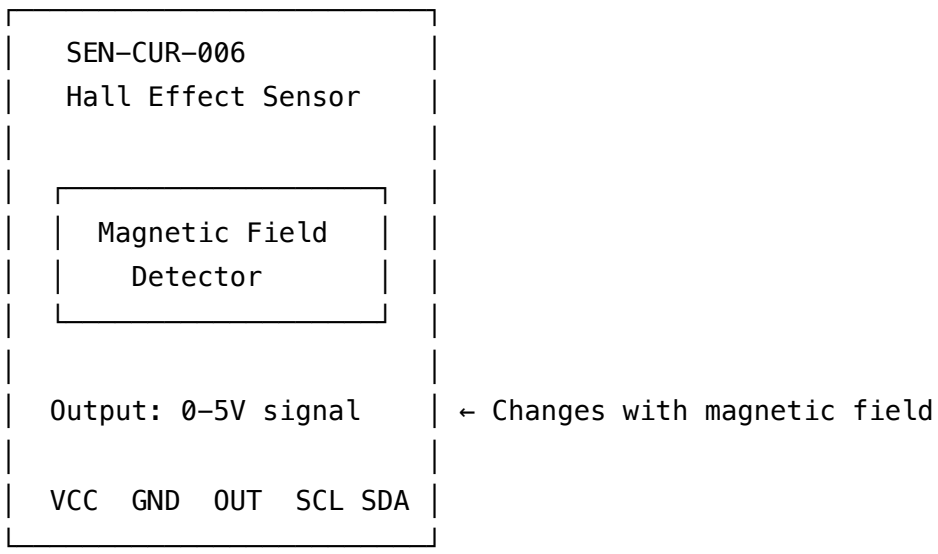
- Where the rescuer's hands are positioned
- The angle of thrust
- Movement during compressions

Output data:

- Roll, pitch, yaw angles (in degrees)
- Acceleration values (in m/s^2)
- Angular velocity (degrees/second)

Think of it as: A compass and level combined - tells you exactly how something is oriented in 3D space.

3. SEN-CUR-006 Hall Effect Sensor - Your Force Sensors



What it does:

- Detects **magnetic fields**
- Stronger field = higher voltage output
- Can measure current flowing through a wire (creates magnetic field)

For your vest:
You'll use these to measure the **force** of thrusts by:

1. Placing a flexible material between two magnetic layers
2. When pressure is applied, magnets get closer together
3. Hall sensor detects the stronger magnetic field
4. Stronger field = more force applied

Output data:

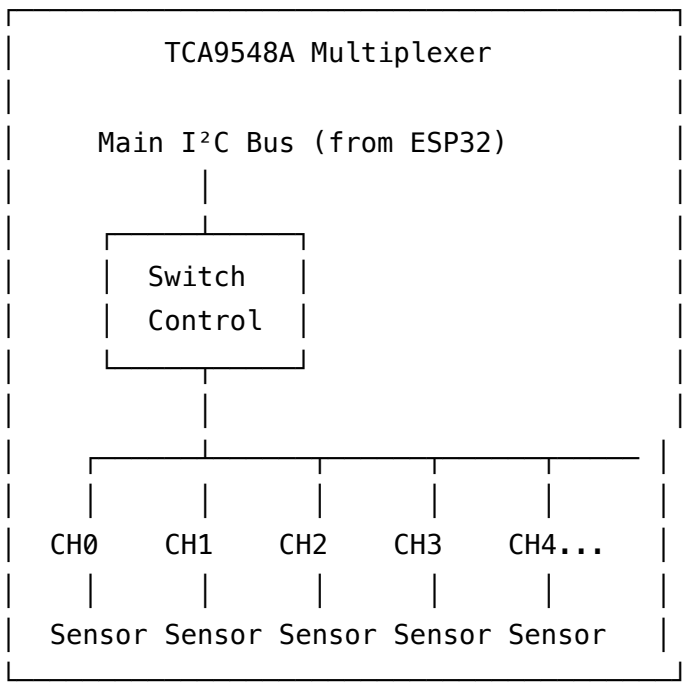
- Voltage (0-3.3V) proportional to magnetic field strength
- Can be calibrated to Newtons of force

Alternative setup (easier for beginners):

- Use the current-sensing capability
- Measure current through a pressure-sensitive resistor
- Current proportional to force applied

Think of it as: A bathroom scale, but for measuring thrust force instead of weight.

4. TCA9548A I²C Multiplexer - Your Traffic Controller



What it does:

- Allows **multiple sensors with the same address** to work together
- Acts like a switchboard - selects which sensor to talk to
- Has 8 channels (CH0-CH7)

Why you need it:

- Problem: Each I²C sensor needs a unique address
- Problem: MPU9250 sensors all have the same address (0x68)
- Solution: Put each MPU9250 on a different channel of the multiplexer

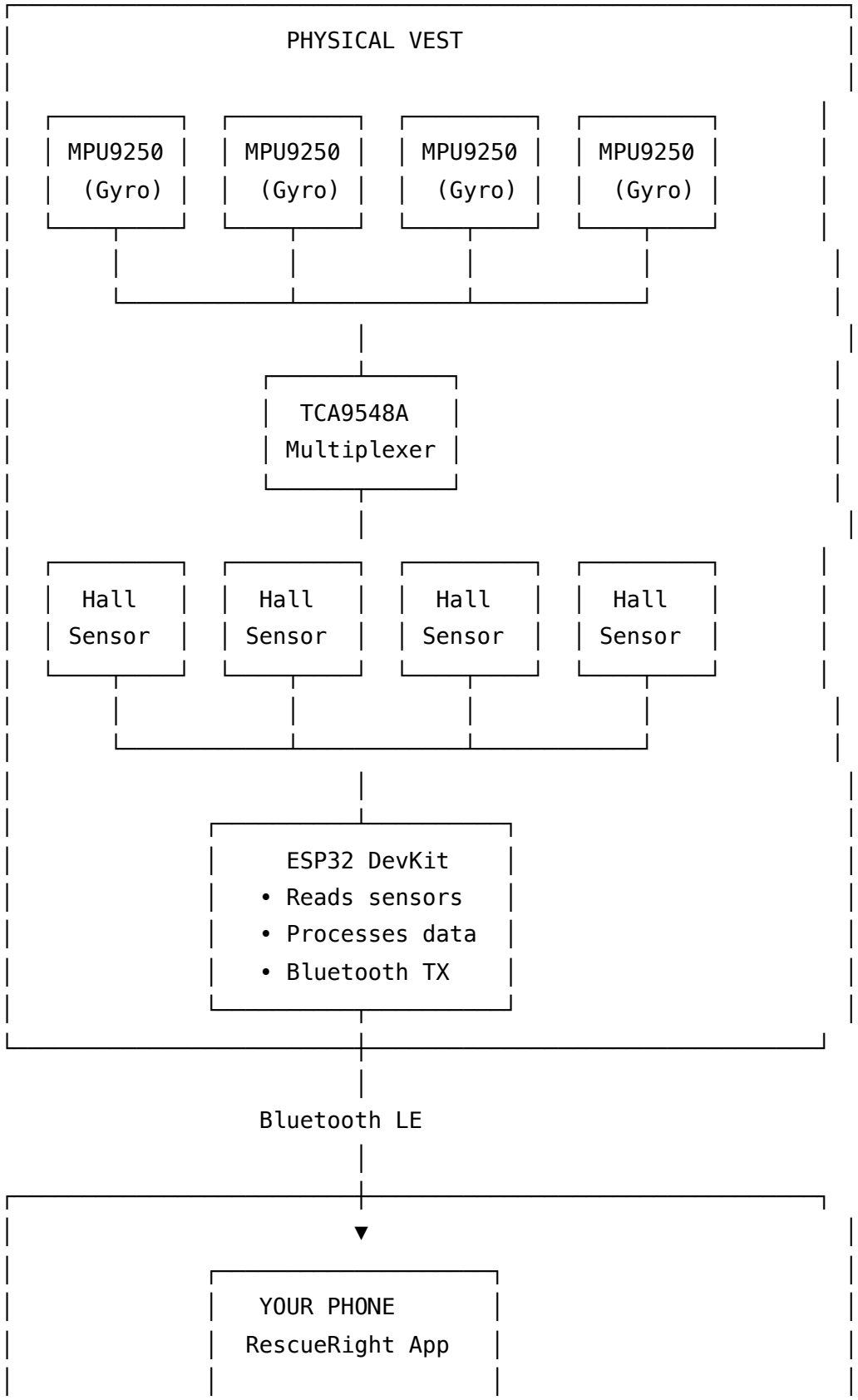
How it works:

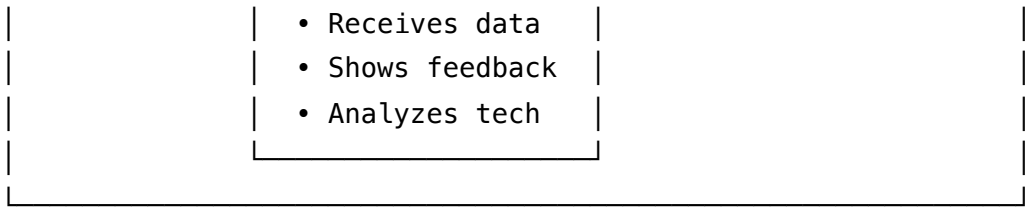
1. ESP32 tells multiplexer: "Open channel 2"
2. Multiplexer connects only channel 2 sensor to the I²C bus
3. ESP32 reads data from that sensor
4. ESP32 tells multiplexer: "Close channel 2, open channel 3"
5. Repeat for all sensors

Think of it as: A telephone switchboard operator connecting different phone lines one at a time.

🔄 How Everything Works Together

System Architecture





Data Flow

1. Trainee performs Heimlich maneuver on vest
   ↓
2. Sensors detect:
   • Hand position (MPU9250s measure orientation)
   • Force applied (Hall sensors measure pressure)
   ↓
3. ESP32 reads sensors every 100ms via I²C
   ↓
4. ESP32 processes data:
   • Converts raw values to meaningful units
   • Applies calibration
   • Packages into Bluetooth format
   ↓
5. ESP32 sends via Bluetooth LE to phone
   ↓
6. Your app receives data and displays:
   • Real-time heatmap of hand position
   • Force gauge showing thrust strength
   • AI feedback card with instructions

⚡ Essential Electronics Concepts

Before we start wiring, let's understand some basics. Think of electricity like water flowing through pipes.

1. Voltage (V) - Electrical Pressure

Water analogy: Water pressure in a pipe

```
Higher Voltage = Higher Pressure

  5V ████████████████  ← High pressure

  3.3V ██████████  ← Medium pressure (ESP32 level)

  0V —  ← No pressure (Ground)
```

Key points:

- ESP32 operates at **3.3V** (not 5V!)
- Connecting 5V directly to ESP32 pins can **destroy it**
- Ground (GND) is 0V - the reference point
- All devices need a common ground

Real example: USB provides 5V, but ESP32 has an internal voltage regulator that converts it to 3.3V for the chip.

2. Current (A or mA) - Flow Rate

Water analogy: How much water flows through the pipe

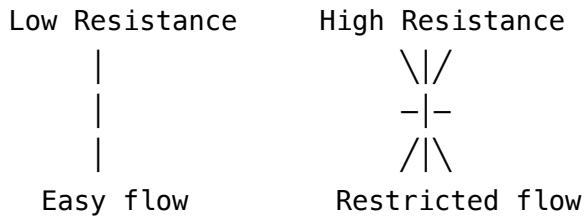
```
1000mA = 1A

Your components:
• ESP32: 500mA maximum
• Each MPU9250: 3.5mA
• Each Hall sensor: 5mA
• Total for your setup: ~50mA (well under limit)
```

Key points:

- Measured in Amperes (A) or milliamperes (mA)
- 1A = 1000mA
- Each component "pulls" current based on its needs
- Power supply must provide enough current for all components

3. Resistance (Ω) - Opposition to Flow

Water analogy: Narrowness of the pipe

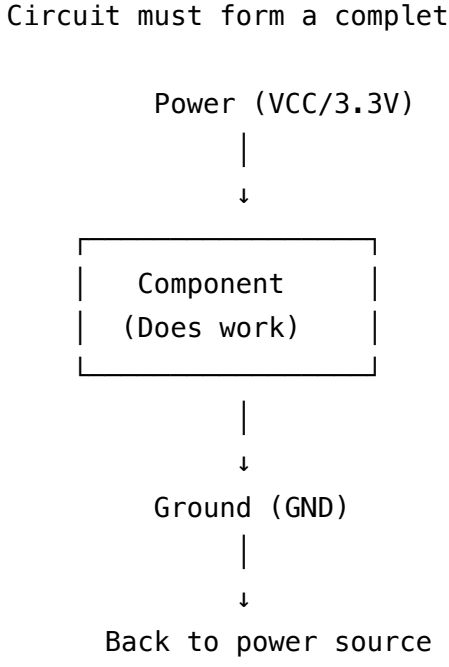


Key points:

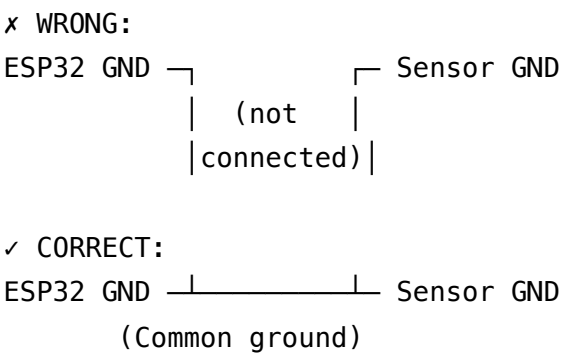
- Measured in Ohms (Ω)
- Used in "pull-up" and "pull-down" resistors for I²C
- Higher resistance = less current flow

For I²C: You need pull-up resistors (usually 4.7kΩ) on SDA and SCL lines.

4. Power and Ground



Critical rule: ALL GROUNDS MUST BE CONNECTED TOGETHER



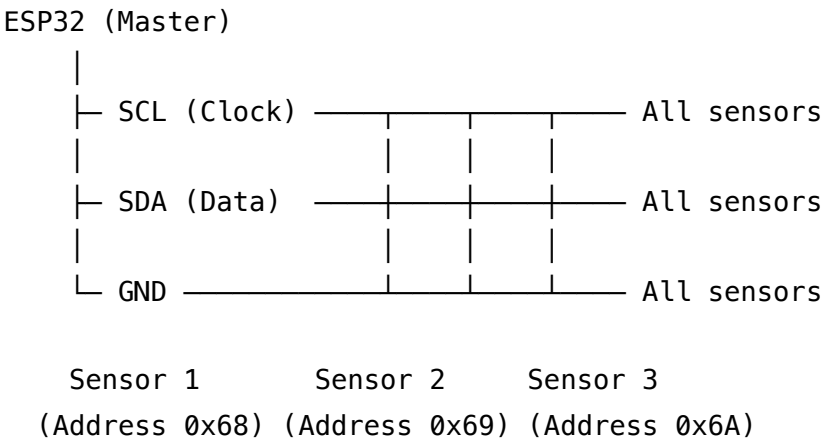
🔌 I²C Communication Deep Dive

I²C (Inter-Integrated Circuit, pronounced "I-squared-C") is how your ESP32 talks to sensors.

What is I²C?

Simple analogy: Think of it as a two-way radio system:

- **One boss (ESP32)** gives orders
- **Multiple workers (sensors)** each have a unique ID (address)
- **Two wires:** One for clock signals (timing), one for data



I²C Pins You'll Use

On ESP32 DevKit V1:

```
Primary I²C Bus:
• GPIO 21 = SDA (Data line)
• GPIO 22 = SCL (Clock line)

Additional I²C buses you can create:
• GPIO 18 & 19 (for your 4 MPUs)
• GPIO 25 & 26 (for your 4 MPUs)
• GPIO 32 & 33 (for your 4 MPUs)
```

Why Multiple I²C Buses?

Problem: All MPU9250 sensors have the same I²C address (0x68)

Solution 1: Use the TCA9548A multiplexer (recommended for beginners)

```
ESP32 GPIO 21 & 22 → TCA9548A → 8 channels → Each sensor on different channel
```

Solution 2: Create separate I²C buses on different GPIO pairs

```
Bus 1 (GPIO 18 & 19) → MPU #1
Bus 2 (GPIO 21 & 22) → MPU #2
Bus 3 (GPIO 25 & 26) → MPU #3
Bus 4 (GPIO 32 & 33) → MPU #4
```

I²C Addresses

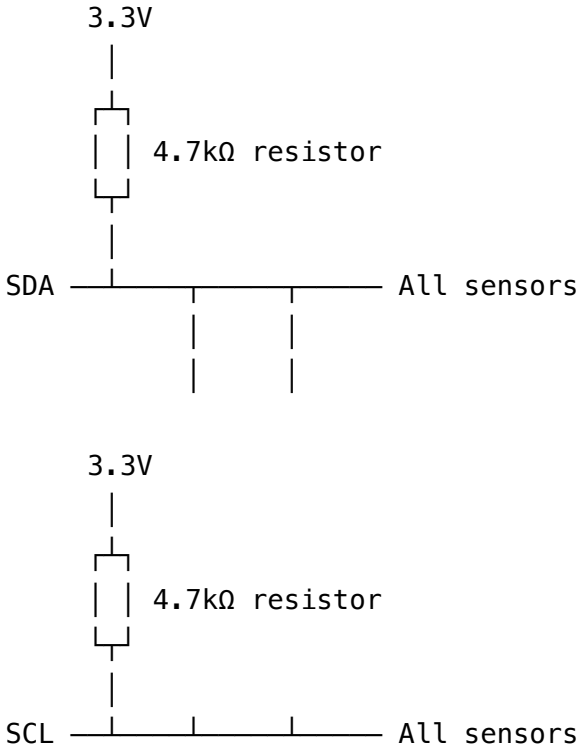
Every I²C device has a unique 7-bit address:

```
Common I²C addresses:
• MPU9250: 0x68 (or 0x69 if AD0 pin is high)
• TCA9548A: 0x70 (default)
• Hall sensor (if I²C): Usually 0x40–0x5F
```

Important: No two devices on the same bus can have the same address!

Pull-Up Resistors

I²C requires pull-up resistors on SDA and SCL lines:



Why needed: I²C uses "open-drain" communication - devices can only pull the line LOW (to GND). Pull-up resistors pull the line HIGH (to 3.3V) when no device is active.

Good news: Many sensor breakout boards have built-in pull-up resistors. You can check by looking for small resistors near the SDA/SCL pins.

🎛️ Your Sensor Setup Explained

Current Configuration

Based on your description:

```
4 MPU9250 (Gyroscopes) connected to:
├─ GPIO 18 & 19 (I²C Bus 1)
├─ GPIO 21 & 22 (I²C Bus 2)
├─ GPIO 25 & 26 (I²C Bus 3)
└─ GPIO 32 & 33 (I²C Bus 4)

4 Hall Sensors connected to:
├─ GPIO 34 (Analog input)
├─ GPIO 35 (Analog input)
├─ GPIO 36 (Analog input)
└─ GPIO 39 (Analog input)
```

Understanding the Connection Strategy

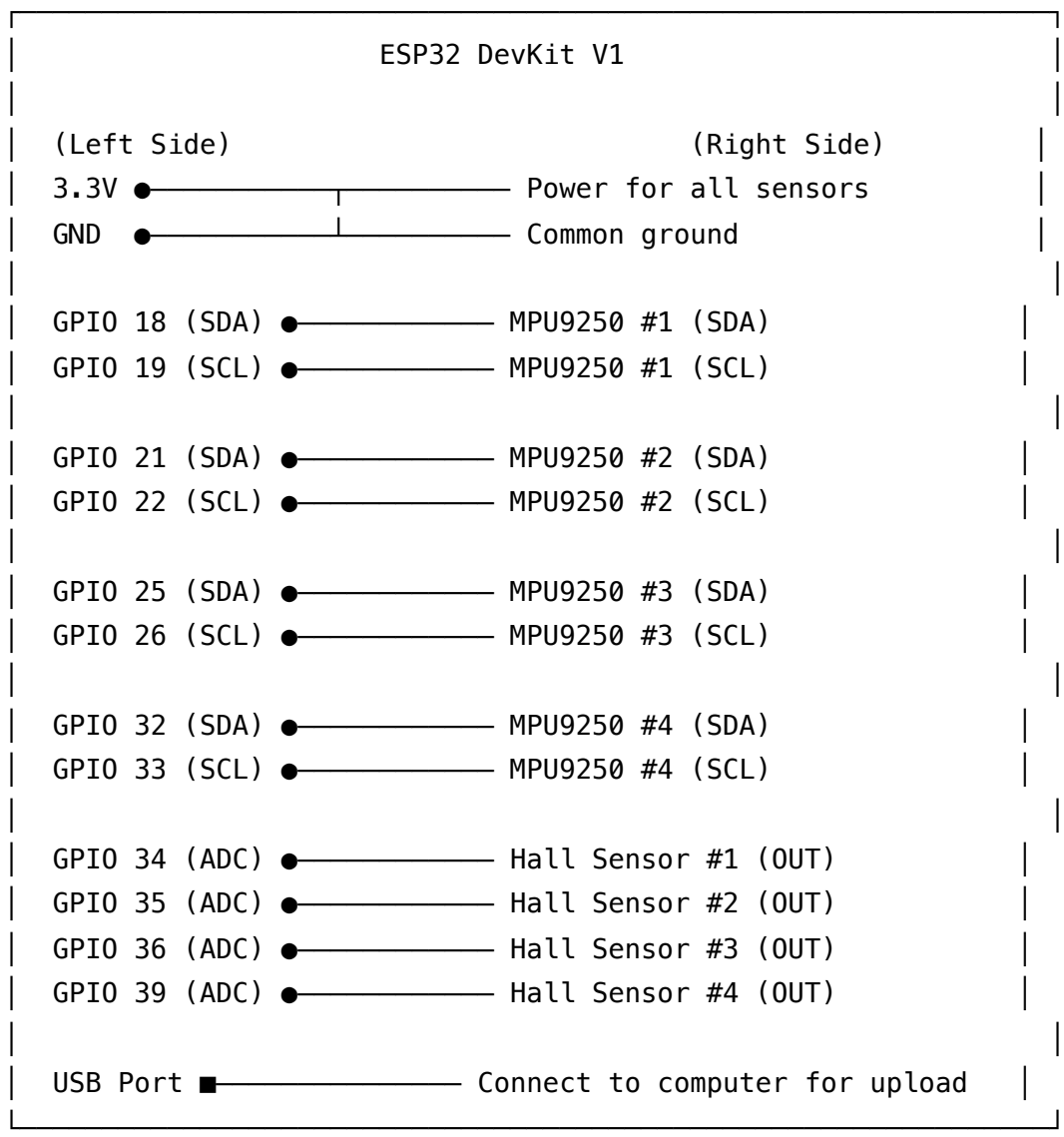
For MPU9250 (I²C sensors):

- You've created 4 separate I²C buses
- Each bus has its own SDA and SCL pins
- This bypasses the address conflict issue
- **Trade-off:** Uses more GPIO pins but simpler code

For Hall Sensors (Analog sensors):

- Connected directly to ADC (Analog-to-Digital Converter) pins
- No I²C needed - just read voltage
- **Note:** GPIOs 34-39 are **input-only** on ESP32 (perfect for sensors)

Wiring Diagram for Your Setup



Each MPU9250:
VCC → ESP32 3.3V
GND → ESP32 GND
SDA → ESP32 GPIO (see above)
SCL → ESP32 GPIO (see above)

Each Hall Sensor (SEN-CUR-006):
VCC → ESP32 3.3V
GND → ESP32 GND
OUT → ESP32 GPIO (34, 35, 36, or 39)

🛠️ Step-by-Step Hardware Assembly

Tools & Materials Needed

Tools:

- Soldering iron (if sensors don't have pins)
- Wire cutters
- Wire strippers
- Multimeter (for testing)
- Small screwdriver
- USB cable (Type-C or Micro-USB for ESP32)

Materials:

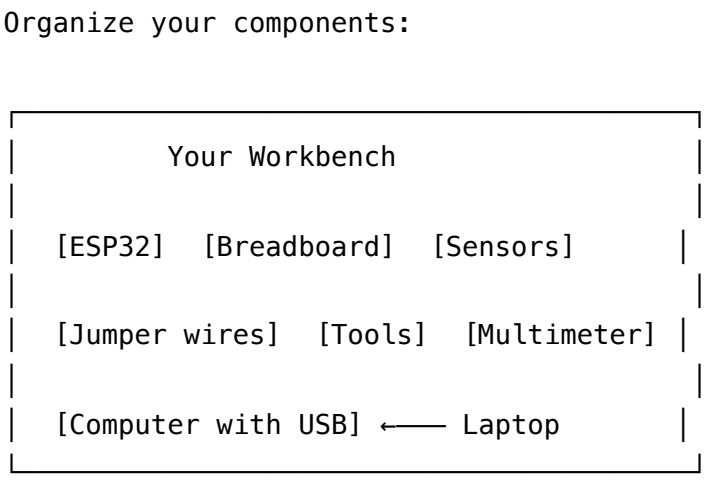
- Breadboard (1x large or 2x small)
- Jumper wires (male-to-male, male-to-female)
- 2x 4.7kΩ resistors (if sensors don't have built-in pull-ups)
- Power supply (USB is fine for testing)

Optional but recommended:

- Heat shrink tubing
- Electrical tape
- Cable ties
- Mounting tape for attaching to vest

Assembly Steps

Step 1: Set Up Your Workspace



Safety first:

1. Work on a non-conductive surface
2. Keep food and drinks away
3. Avoid touching components while powered
4. Have good lighting

Step 2: Prepare the ESP32

Check your board:

```
ESP32 DevKit V1 - Visual Inspection:

1. Find the USB port (Type-C or Micro-USB)
   └─ This is how you'll power and program it

2. Locate the EN (Enable) button
   └─ Press to reset the board

3. Locate the BOOT button
   └─ Hold while pressing EN to enter programming mode

4. Check all pins:
   └─ 38 pins total (19 on each side)
   └─ Should be straight, not bent

5. Look for LED indicators:
   └─ Power LED (should light when connected to USB)
   └─ Programmable LED (usually on GPIO 2)
```

Test the ESP32:

1. Connect ESP32 to computer via USB
2. Power LED should light up
3. If using Mac/Linux: Open Terminal and type `ls /dev/tty.*`
4. You should see something like `/dev/tty.usbserial-0001`
5. If using Windows: Check Device Manager → Ports (COM & LPT)

If ESP32 doesn't appear:

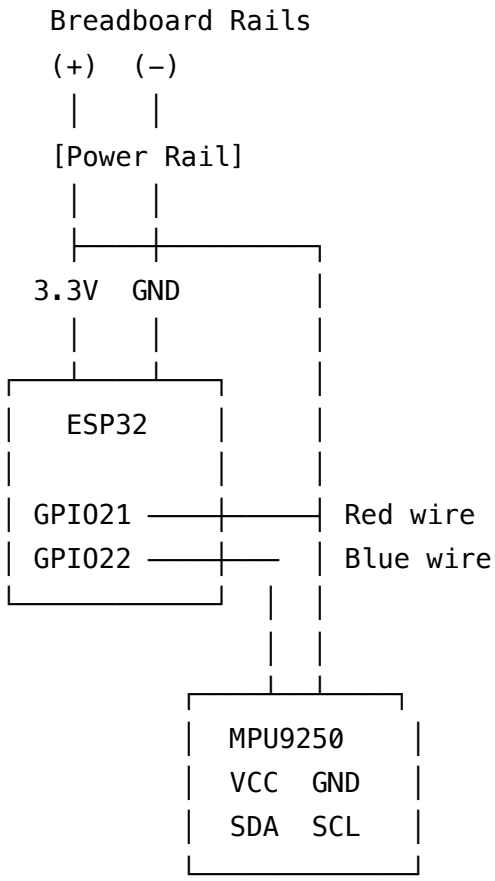
- Try a different USB cable (must be data cable, not just power)
- Install CH340 or CP2102 drivers (Google "ESP32 USB driver" + your OS)

Step 3: Wire One MPU9250 (Test First!)

Start simple - wire just ONE sensor first:

ESP32 Pin	→	MPU9250 Pin
3.3V	→	VCC
GND	→	GND
GPIO 21 (SDA)	→	SDA
GPIO 22 (SCL)	→	SCL

Physical wiring using breadboard:



Step-by-step visual process:

1. **Insert ESP32 into breadboard**
   - Straddle the center gap
   - Press firmly but gently
   - Pins should be fully inserted
2. **Connect power rails**
   - Red jumper: ESP32 3.3V pin → Breadboard (+) rail
   - Black jumper: ESP32 GND pin → Breadboard (-) rail
3. **Insert MPU9250**

- Place in a different row from ESP32
- Leave space between components

4. **Connect MPU9250 power**
   - Red jumper: MPU VCC → Breadboard (+) rail
   - Black jumper: MPU GND → Breadboard (-) rail
5. **Connect I²C data lines**
   - Blue jumper: ESP32 GPIO 21 → MPU9250 SDA
   - Green jumper: ESP32 GPIO 22 → MPU9250 SCL

Step 4: Test Your First Sensor

Before wiring all sensors, let's test that this one works!

Install Arduino IDE:

1. Download from: https://www.arduino.cc/en/software
2. Install for your OS (Windows/Mac/Linux)
3. Open Arduino IDE

Install ESP32 board support:

1. Go to **File** → **Preferences**
2. In "Additional Board Manager URLs", paste:

```
https://dl.espressif.com/dl/package_esp32_index.json
```

3. Click OK
4. Go to **Tools** → **Board** → **Boards Manager**
5. Search "ESP32"
6. Install "esp32 by Espressif Systems"
7. Wait for installation (may take 5-10 minutes)

Configure Arduino IDE for ESP32:

1. Go to **Tools** → **Board** → **ESP32 Arduino**
2. Select "ESP32 Dev Module"
3. Go to **Tools** → **Port**
4. Select your ESP32 port (e.g., /dev/tty.usbserial-XXXX)
5. Set **Upload Speed** to 115200

Install MPU9250 library:

1. Go to **Sketch → Include Library → Manage Libraries**
2. Search "MPU9250"
3. Install "MPU9250 by hideakitai" (recommended)
4. Click Install

Upload test code:

```
#include <Wire.h>
#include <MPU9250.h>

// Create MPU9250 object
MPU9250 mpu;

void setup() {
  // Start serial communication
  Serial.begin(115200);
  delay(1000);

  Serial.println("RescueRight - MPU9250 Test");
  Serial.println("————————————————————————————");

  // Initialize I²C on GPIO 21 (SDA) and GPIO 22 (SCL)
  Wire.begin(21, 22);

  // Initialize MPU9250
  if (!mpu.setup(0x68)) {
    Serial.println("❌ MPU9250 connection failed!");
    Serial.println("Check your wiring:");
    Serial.println("  • VCC → 3.3V");
    Serial.println("  • GND → GND");
    Serial.println("  • SDA → GPIO 21");
    Serial.println("  • SCL → GPIO 22");
    while (1) {
      delay(1000);
    }
  }

  Serial.println("✓ MPU9250 connected successfully!");
  Serial.println("\nStarting measurements...\n");
}

void loop() {
  // Update sensor readings
  if (mpu.update()) {
    // Print orientation (Euler angles)
    Serial.print("Roll: ");
    Serial.print(mpu.getRoll(), 1);
    Serial.print("° | Pitch: ");
    Serial.print(mpu.getPitch(), 1);
    Serial.print("° | Yaw: ");
```

```
    Serial.print(mpu.getYaw(), 1);
    Serial.println("°");
  }

  delay(100); // Read 10 times per second
}
```

Upload and test:

1. Click the **Upload** button (→) in Arduino IDE
2. Wait for "Connecting..." message
3. If it says "Failed to connect", hold BOOT button while clicking upload
4. Wait for "Done uploading"
5. Open **Tools → Serial Monitor**
6. Set baud rate to **115200**

Expected output:

```
RescueRight - MPU9250 Test
_____________________________

✓ MPU9250 connected successfully!

Starting measurements...

Roll: 2.3° | Pitch: -1.5° | Yaw: 45.2°
Roll: 2.4° | Pitch: -1.4° | Yaw: 45.3°
Roll: 2.3° | Pitch: -1.6° | Yaw: 45.1°
...
```

Try moving the sensor:

- Tilt it forward/back → Pitch changes
- Tilt it left/right → Roll changes
- Rotate it → Yaw changes

If you see this, congratulations! 🎉 Your first sensor works!

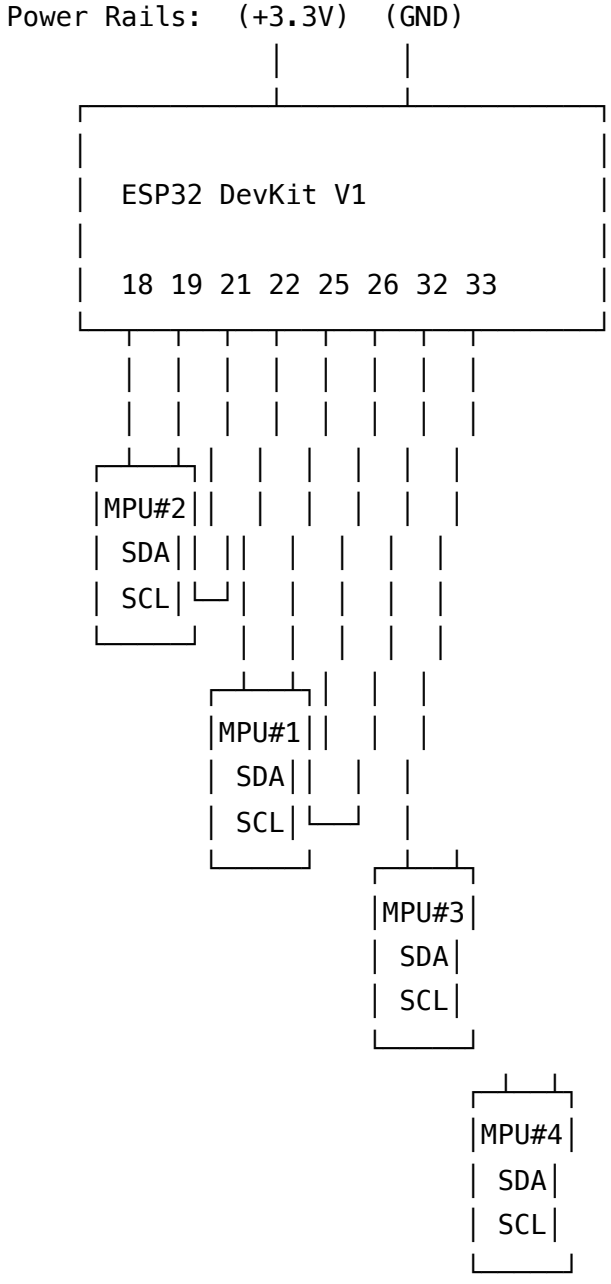
Step 5: Add Remaining MPU9250 Sensors

Now that one works, let's add the other three.

Wiring diagram for all 4 MPUs:

```
MPU #1: GPIO 21 (SDA) + GPIO 22 (SCL)  ← Already done
MPU #2: GPIO 18 (SDA) + GPIO 19 (SCL)
MPU #3: GPIO 25 (SDA) + GPIO 26 (SCL)
MPU #4: GPIO 32 (SDA) + GPIO 33 (SCL)

All MPUs share:
• 3.3V power rail
• GND rail
```

Physical layout on breadboard:



Connection table:

MPU #	VCC	GND	SDA	SCL
#1	3.3V	GND	GPIO 21	GPIO 22
#2	3.3V	GND	GPIO 18	GPIO 19
#3	3.3V	GND	GPIO 25	GPIO 26
#4	3.3V	GND	GPIO 32	GPIO 33

Color coding suggestion:

- Red wires: VCC (3.3V)
- Black wires: GND
- Blue wires: SDA
- Green wires: SCL

Step 6: Test All MPU9250 Sensors

Upload this test code:

```
#include <Wire.h>
#include <MPU9250.h>

// Create 4 MPU9250 objects
MPU9250 mpu1, mpu2, mpu3, mpu4;

// Define I²C pins for each bus
const int SDA1 = 21, SCL1 = 22;
const int SDA2 = 18, SCL2 = 19;
const int SDA3 = 25, SCL3 = 26;
const int SDA4 = 32, SCL4 = 33;

// Create TwoWire objects for each I²C bus
TwoWire I2C1 = TwoWire(0);
TwoWire I2C2 = TwoWire(1);
TwoWire I2C3 = TwoWire(2);
TwoWire I2C4 = TwoWire(3);

void setup() {
  Serial.begin(115200);
  delay(2000);

  Serial.println("RescueRight - Testing All 4 MPU9250 Sensors");
  Serial.println("═══════════════════════════════════════════════════");

  // Initialize all I²C buses
  I2C1.begin(SDA1, SCL1, 400000);
  I2C2.begin(SDA2, SCL2, 400000);
  I2C3.begin(SDA3, SCL3, 400000);
  I2C4.begin(SDA4, SCL4, 400000);

  // Initialize MPU sensors
  Serial.println("Initializing sensors...\n");

  // MPU #1
  Serial.print("MPU #1 (GPIO 21/22): ");
  if (mpu1.setup(0x68, MPU9250Setting(), I2C1)) {
    Serial.println("✓ Connected");
  } else {
    Serial.println("❌ Failed");
  }

  // MPU #2
```

```
  Serial.print("MPU #2 (GPIO 18/19): ");
  if (mpu2.setup(0x68, MPU9250Setting(), I2C2)) {
    Serial.println("✓ Connected");
  } else {
    Serial.println("❌ Failed");
  }

  // MPU #3
  Serial.print("MPU #3 (GPIO 25/26): ");
  if (mpu3.setup(0x68, MPU9250Setting(), I2C3)) {
    Serial.println("✓ Connected");
  } else {
    Serial.println("❌ Failed");
  }

  // MPU #4
  Serial.print("MPU #4 (GPIO 32/33): ");
  if (mpu4.setup(0x68, MPU9250Setting(), I2C4)) {
    Serial.println("✓ Connected");
  } else {
    Serial.println("❌ Failed");
  }

  Serial.println("\n──────────────────────────────────────────────────────────");
  Serial.println("Reading data from all sensors...\n");
  delay(1000);
}

void loop() {
  // Update all sensors
  mpu1.update();
  mpu2.update();
  mpu3.update();
  mpu4.update();

  // Print readings from all sensors
  Serial.println("┌──────────────────────────────────────────────────────┐");

  Serial.print("│ MPU #1: ");
  printOrientation(mpu1);

  Serial.print("│ MPU #2: ");
  printOrientation(mpu2);
```

```
  Serial.print("│ MPU #3: ");
  printOrientation(mpu3);

  Serial.print("│ MPU #4: ");
  printOrientation(mpu4);

  Serial.println("└──────────────────────────────────────────────────────┘\n");

  delay(500); // Update every 500ms for easier reading
}

void printOrientation(MPU9250 &mpu) {
  Serial.print("R:");
  Serial.print(mpu.getRoll(), 1);
  Serial.print("° P:");
  Serial.print(mpu.getPitch(), 1);
  Serial.print("° Y:");
  Serial.print(mpu.getYaw(), 1);
  Serial.println("° │");
}
```

Expected output:

```
RescueRight - Testing All 4 MPU9250 Sensors
═══════════════════════════════════════════════════
Initializing sensors...

MPU #1 (GPIO 21/22): ✓ Connected
MPU #2 (GPIO 18/19): ✓ Connected
MPU #3 (GPIO 25/26): ✓ Connected
MPU #4 (GPIO 32/33): ✓ Connected

───────────────────────────────────────────────────
Reading data from all sensors...

┌──────────────────────────────────────────────────┐
│ MPU #1: R:2.3° P:-1.5° Y:45.2° │
│ MPU #2: R:3.1° P:0.8° Y:12.4°  │
│ MPU #3: R:-0.5° P:2.3° Y:89.1° │
│ MPU #4: R:1.2° P:-0.3° Y:23.7° │
└──────────────────────────────────────────────────┘
```

Try moving each sensor individually to verify they're all responding correctly!

Step 7: Add Hall Effect Sensors

Now let's add the force sensors.

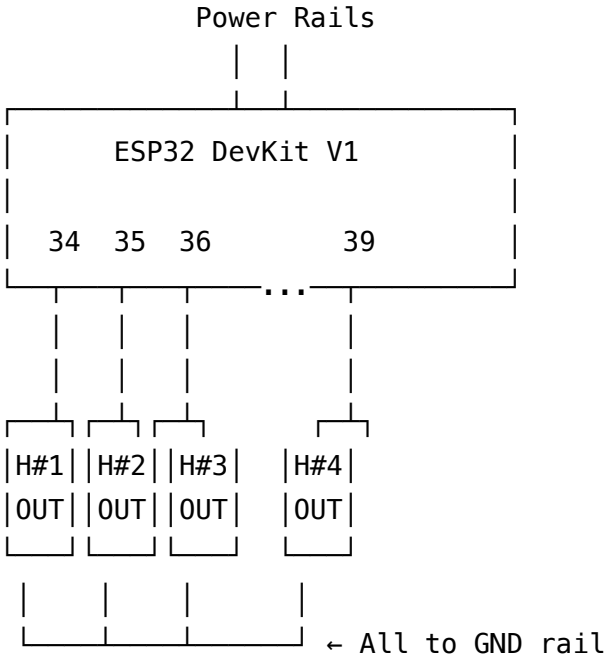
Hall sensor wiring:

```
Hall Sensor #1:
VCC  → ESP32 3.3V
GND  → ESP32 GND
OUT  → ESP32 GPIO 34

Hall Sensor #2:
VCC  → ESP32 3.3V
GND  → ESP32 GND
OUT  → ESP32 GPIO 35

Hall Sensor #3:
VCC  → ESP32 3.3V
GND  → ESP32 GND
OUT  → ESP32 GPIO 36

Hall Sensor #4:
VCC  → ESP32 3.3V
GND  → ESP32 GND
OUT  → ESP32 GPIO 39
```

Important notes about GPIO 34-39:

- These are **input-only** pins (no OUTPUT mode)
- They support **analog reading** (ADC)
- Perfect for sensors!
- **No internal pull-up/pull-down resistors**

Physical setup:



Step 8: Test Hall Sensors

Upload this test code:

```
// Define Hall sensor pins (GPIO 34, 35, 36, 39)
const int HALL1_PIN = 34;
const int HALL2_PIN = 35;
const int HALL3_PIN = 36;
const int HALL4_PIN = 39;

// Store baseline values (calibration)
int baseline1, baseline2, baseline3, baseline4;

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("RescueRight - Hall Sensor Test");
  Serial.println("═══════════════════════════════════════");

  // Configure ADC resolution (12-bit = 0-4095)
  analogReadResolution(12);

  // Calibrate - read baseline values
  Serial.println("Calibrating... (don't press anything)");
  delay(2000);

  baseline1 = analogRead(HALL1_PIN);
  baseline2 = analogRead(HALL2_PIN);
  baseline3 = analogRead(HALL3_PIN);
  baseline4 = analogRead(HALL4_PIN);

  Serial.println("Baseline values:");
  Serial.print("  Sensor 1: "); Serial.println(baseline1);
  Serial.print("  Sensor 2: "); Serial.println(baseline2);
  Serial.print("  Sensor 3: "); Serial.println(baseline3);
  Serial.print("  Sensor 4: "); Serial.println(baseline4);
  Serial.println("\nReading force values...\n");
}

void loop() {
  // Read current values
  int value1 = analogRead(HALL1_PIN);
  int value2 = analogRead(HALL2_PIN);
  int value3 = analogRead(HALL3_PIN);
  int value4 = analogRead(HALL4_PIN);
```

```
  // Calculate change from baseline (indicates force)
  int force1 = abs(value1 - baseline1);
  int force2 = abs(value2 - baseline2);
  int force3 = abs(value3 - baseline3);
  int force4 = abs(value4 - baseline4);

  // Convert to approximate Newtons (calibration needed)
  // This is a rough estimate - you'll refine this later
  float newtons1 = force1 * 0.05;
  float newtons2 = force2 * 0.05;
  float newtons3 = force3 * 0.05;
  float newtons4 = force4 * 0.05;

  // Display results
  Serial.println("┌─────────────────────────────────────────┐");
  Serial.print("│ Hall #1: ");
  printForce(newtons1);
  Serial.print("│ Hall #2: ");
  printForce(newtons2);
  Serial.print("│ Hall #3: ");
  printForce(newtons3);
  Serial.print("│ Hall #4: ");
  printForce(newtons4);
  Serial.println("└─────────────────────────────────────────┘\n");

  delay(200); // Read 5 times per second
}

void printForce(float newtons) {
  // Print with visual force indicator
  Serial.print(newtons, 1);
  Serial.print("N ");

  // Visual bar graph
  int bars = (int)(newtons / 2); // Each bar = 2N
  for (int i = 0; i < bars && i < 50; i++) {
    Serial.print("█");
  }

  // Feedback
  if (newtons < 50) {
    Serial.println(" (Too light)        │");
  } else if (newtons > 150) {
```

```
    Serial.println(" (TOO STRONG! RISK!)|");
  } else if (newtons >= 80 && newtons <= 120) {
    Serial.println(" ✓ PERFECT!         |");
  } else {
    Serial.println("                    |");
  }
}
```

Testing Hall sensors:

1. Upload the code
2. Open Serial Monitor
3. Wait for calibration
4. Try applying pressure near each Hall sensor:
    - Use a magnet if available
    - Or press gently if in a force-sensitive setup
5. Watch the readings change

Expected output (example):

```
RescueRight - Hall Sensor Test
═══════════════════════════════════════

Calibrating... (don't press anything)
Baseline values:
  Sensor 1: 2048
  Sensor 2: 2052
  Sensor 3: 2045
  Sensor 4: 2050

Reading force values...

┌─────────────────────────────────────────┐
| Hall #1: 95.5N ████████████████████████ ✓ PERFECT!         |
| Hall #2: 25.2N ████████ (Too light)        |
| Hall #3: 165.8N ████████████████████████████████████ (TOO STRONG! RISK!)|
| Hall #4: 0.0N  (Too light)        |
└─────────────────────────────────────────┘
```

Step 9: Final Calibration

For Hall sensors, you need to calibrate force readings:

1. **Get a known weight** (e.g., 1kg = ~10N, 5kg = ~50N)
2. **Apply weight** to each sensor
3. **Record ADC values**
4. **Calculate conversion factor**:

```
Conversion factor = Known force (N) / ADC change
```

Example calibration:

```
// If 100N force causes ADC to change by 500:
float forceInNewtons = (adcValue - baseline) * (100.0 / 500.0);
// Simplifies to:
float forceInNewtons = (adcValue - baseline) * 0.2;
```

For MPU9250 sensors, you need to:

1. **Mount them** on your vest in their final positions
2. **Run calibration** routine (built into MPU9250 library)
3. **Save offsets** to use in your main code

💻 Programming the ESP32

Complete ESP32 Code for Your MVP

Here's the full code that reads all sensors and sends data via Bluetooth:

```
// ═══════════════════════════════════════════════════════════
//  RescueRight Smart Vest - ESP32 Firmware
//  Version: 1.0 MVP
//  IDEATE 2025 - Team Kiwi
// ═══════════════════════════════════════════════════════════

#include <Wire.h>
#include <MPU9250.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

// ─────────────────────────────────────────────────────────
// CONFIGURATION
// ─────────────────────────────────────────────────────────

// BLE UUIDs (must match your app!)
#define SERVICE_UUID        "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define FORCE_CHAR_UUID     "beb5483e-36e1-4688-b7f5-ea07361b26a8"
#define POSITION_CHAR_UUID  "beb5483e-36e1-4688-b7f5-ea07361b26a9"
#define ANGLE_CHAR_UUID     "beb5483e-36e1-4688-b7f5-ea07361b26aa"

// I²C Pin Definitions
const int SDA1 = 21, SCL1 = 22;  // MPU #1
const int SDA2 = 18, SCL2 = 19;  // MPU #2
const int SDA3 = 25, SCL3 = 26;  // MPU #3
const int SDA4 = 32, SCL4 = 33;  // MPU #4

// Hall Sensor Pins
const int HALL1_PIN = 34;
const int HALL2_PIN = 35;
const int HALL3_PIN = 36;
const int HALL4_PIN = 39;

// Calibration values (you'll set these after testing)
int hallBaseline1, hallBaseline2, hallBaseline3, hallBaseline4;
float hallToNewtons = 0.2;  // Conversion factor (adjust after calibration)

// ─────────────────────────────────────────────────────────
// GLOBAL OBJECTS
// ─────────────────────────────────────────────────────────
```

```
// MPU9250 sensors
MPU9250 mpu1, mpu2, mpu3, mpu4;
TwoWire I2C1 = TwoWire(0);
TwoWire I2C2 = TwoWire(1);
TwoWire I2C3 = TwoWire(2);
TwoWire I2C4 = TwoWire(3);

// BLE objects
BLEServer* pServer = NULL;
BLECharacteristic* pForceCharacteristic = NULL;
BLECharacteristic* pPositionCharacteristic = NULL;
BLECharacteristic* pAngleCharacteristic = NULL;

bool deviceConnected = false;
bool oldDeviceConnected = false;

// ─────────────────────────────────────────────────────────
// BLE CALLBACKS
// ─────────────────────────────────────────────────────────

class ServerCallbacks: public BLEServerCallbacks {
  void onConnect(BLEServer* pServer) {
    deviceConnected = true;
    Serial.println("📱 Device connected");
  }

  void onDisconnect(BLEServer* pServer) {
    deviceConnected = false;
    Serial.println("📱 Device disconnected");
  }
};

// ─────────────────────────────────────────────────────────
// SETUP
// ─────────────────────────────────────────────────────────

void setup() {
  Serial.begin(115200);
  delay(2000);

  Serial.println("═══════════════════════════════════════════");
  Serial.println("  RescueRight Smart Vest - Starting Up");
  Serial.println("  IDEATE 2025 - Team Kiwi");
```

```
Serial.println("═══════════════════════════════════════════\n");

// ─── Initialize MPU9250 Sensors ───
Serial.println("[1/4] Initializing MPU9250 sensors...");

I2C1.begin(SDA1, SCL1, 400000);
I2C2.begin(SDA2, SCL2, 400000);
I2C3.begin(SDA3, SCL3, 400000);
I2C4.begin(SDA4, SCL4, 400000);

if (!mpu1.setup(0x68, MPU9250Setting(), I2C1)) {
  Serial.println("  ❌ MPU #1 failed");
} else {
  Serial.println("  ✓ MPU #1 ready");
}

if (!mpu2.setup(0x68, MPU9250Setting(), I2C2)) {
  Serial.println("  ❌ MPU #2 failed");
} else {
  Serial.println("  ✓ MPU #2 ready");
}

if (!mpu3.setup(0x68, MPU9250Setting(), I2C3)) {
  Serial.println("  ❌ MPU #3 failed");
} else {
  Serial.println("  ✓ MPU #3 ready");
}

if (!mpu4.setup(0x68, MPU9250Setting(), I2C4)) {
  Serial.println("  ❌ MPU #4 failed");
} else {
  Serial.println("  ✓ MPU #4 ready");
}

// ─── Initialize Hall Sensors ───
Serial.println("\n[2/4] Calibrating Hall sensors...");
analogReadResolution(12);

delay(1000);
hallBaseline1 = analogRead(HALL1_PIN);
hallBaseline2 = analogRead(HALL2_PIN);
hallBaseline3 = analogRead(HALL3_PIN);
hallBaseline4 = analogRead(HALL4_PIN);
```

```
Serial.println("  ✓ Hall sensors calibrated");

// ─── Initialize Bluetooth ───
Serial.println("\n[3/4] Starting Bluetooth...");

BLEDevice::init("RescueRight Vest #001");
pServer = BLEDevice::createServer();
pServer->setCallbacks(new ServerCallbacks());

BLEService *pService = pServer->createService(SERVICE_UUID);

// Force characteristic
pForceCharacteristic = pService->createCharacteristic(
  FORCE_CHAR_UUID,
  BLECharacteristic::PROPERTY_READ |
  BLECharacteristic::PROPERTY_NOTIFY
);
pForceCharacteristic->addDescriptor(new BLE2902());

// Position characteristic
pPositionCharacteristic = pService->createCharacteristic(
  POSITION_CHAR_UUID,
  BLECharacteristic::PROPERTY_READ |
  BLECharacteristic::PROPERTY_NOTIFY
);
pPositionCharacteristic->addDescriptor(new BLE2902());

// Angle characteristic
pAngleCharacteristic = pService->createCharacteristic(
  ANGLE_CHAR_UUID,
  BLECharacteristic::PROPERTY_READ |
  BLECharacteristic::PROPERTY_NOTIFY
);
pAngleCharacteristic->addDescriptor(new BLE2902());

pService->start();

BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
pAdvertising->addServiceUUID(SERVICE_UUID);
pAdvertising->setScanResponse(true);
BLEDevice::startAdvertising();
```

```
  Serial.println("  ✓ Bluetooth ready");
  Serial.println("  📡 Advertising as 'RescueRight Vest #001'");

  // ─── Ready! ───
  Serial.println("\n[4/4] System ready!");
  Serial.println("═══════════════════════════════════════════════════");
  Serial.println("Waiting for connection...\n");
}

// ═══════════════════════════════════════════════════════════════
// MAIN LOOP
// ═══════════════════════════════════════════════════════════════

void loop() {
  // Handle connection status changes
  if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = deviceConnected;
    Serial.println("Starting data transmission...\n");
  }

  if (!deviceConnected && oldDeviceConnected) {
    delay(500);
    pServer->startAdvertising();
    Serial.println("Restarting advertising...\n");
    oldDeviceConnected = deviceConnected;
  }

  // Only send data if connected
  if (deviceConnected) {
    // ─── Read Sensors ───

    // Update MPU9250 sensors
    mpu1.update();
    mpu2.update();
    mpu3.update();
    mpu4.update();

    // Read Hall sensors
    int hall1 = analogRead(HALL1_PIN);
    int hall2 = analogRead(HALL2_PIN);
    int hall3 = analogRead(HALL3_PIN);
    int hall4 = analogRead(HALL4_PIN);
```

```
    // ─── Calculate Force ───
    float force1 = abs(hall1 - hallBaseline1) * hallToNewtons;
    float force2 = abs(hall2 - hallBaseline2) * hallToNewtons;
    float force3 = abs(hall3 - hallBaseline3) * hallToNewtons;
    float force4 = abs(hall4 - hallBaseline4) * hallToNewtons;

    // Total force (sum of all sensors)
    float totalForce = force1 + force2 + force3 + force4;

    // ─── Calculate Hand Position ───
    // Average position from MPU sensors
    // (You'll refine this algorithm based on vest layout)
    float avgRoll = (mpu1.getRoll() + mpu2.getRoll() +
                     mpu3.getRoll() + mpu4.getRoll()) / 4.0;
    float avgPitch = (mpu1.getPitch() + mpu2.getPitch() +
                      mpu3.getPitch() + mpu4.getPitch()) / 4.0;

    // Convert to normalized 0-1 position
    // (This mapping depends on your vest geometry)
    float posX = 0.5 + (avgRoll / 90.0) * 0.5;   // Roll → X position
    float posY = 0.5 - (avgPitch / 90.0) * 0.5;  // Pitch → Y position

    // Clamp to 0-1 range
    posX = constrain(posX, 0.0, 1.0);
    posY = constrain(posY, 0.0, 1.0);

    // ─── Calculate Thrust Angle ───
    float avgYaw = (mpu1.getYaw() + mpu2.getYaw() +
                    mpu3.getYaw() + mpu4.getYaw()) / 4.0;

    // ─── Send via Bluetooth ───
    sendForceData(totalForce);
    sendPositionData(posX, posY);
    sendAngleData(avgYaw);

    // ─── Debug Output ───
    printDebugInfo(totalForce, posX, posY, avgYaw);

    // Update rate: 10Hz
    delay(100);
  }
}
```

```
// ─────────────────────────────────────────────────────────
// HELPER FUNCTIONS
// ─────────────────────────────────────────────────────────

void sendForceData(float force) {
  uint8_t data[4];
  memcpy(data, &force, 4);
  pForceCharacteristic->setValue(data, 4);
  pForceCharacteristic->notify();
}

void sendPositionData(float x, float y) {
  uint8_t data[8];
  memcpy(data, &x, 4);
  memcpy(data + 4, &y, 4);
  pPositionCharacteristic->setValue(data, 8);
  pPositionCharacteristic->notify();
}

void sendAngleData(float angle) {
  uint8_t data[4];
  memcpy(data, &angle, 4);
  pAngleCharacteristic->setValue(data, 4);
  pAngleCharacteristic->notify();
}

void printDebugInfo(float force, float x, float y, float angle) {
  // Only print every 10 cycles to avoid flooding serial
  static int counter = 0;
  if (counter++ % 10 == 0) {
    Serial.print("Force: ");
    Serial.print(force, 1);
    Serial.print("N | Pos: (");
    Serial.print(x, 2);
    Serial.print(", ");
    Serial.print(y, 2);
    Serial.print(") | Angle: ");
    Serial.print(angle, 1);
    Serial.println("°");
  }
}
```

Uploading the Code

1. **Open Arduino IDE**
2. **Copy the complete code** above
3. **Paste into a new sketch**
4. **Save as** "RescueRight_MVP"
5. **Select board:** ESP32 Dev Module
6. **Select port:** Your ESP32 port
7. **Click Upload** (→)
8. **Wait for** "Done uploading"
9. **Open Serial Monitor** (115200 baud)

Expected serial output:

```
═══════════════════════════════════════
  RescueRight Smart Vest – Starting Up
  IDEATE 2025 – Team Kiwi
═══════════════════════════════════════

[1/4] Initializing MPU9250 sensors...
  ✓ MPU #1 ready
  ✓ MPU #2 ready
  ✓ MPU #3 ready
  ✓ MPU #4 ready

[2/4] Calibrating Hall sensors...
  ✓ Hall sensors calibrated

[3/4] Starting Bluetooth...
  ✓ Bluetooth ready
  📡 Advertising as 'RescueRight Vest #001'

[4/4] System ready!
═══════════════════════════════════════
Waiting for connection...

📱 Device connected
Starting data transmission...

Force: 85.3N | Pos: (0.52, 0.48) | Angle: 2.5°
Force: 91.7N | Pos: (0.50, 0.45) | Angle: 1.8°
...
```

🧪 Testing Your Hardware

Test Plan Overview

```
Test 1: Power & Connectivity
    ↓
Test 2: Individual Sensors
    ↓
Test 3: Multiple Sensors Together
    ↓
Test 4: Bluetooth Communication
    ↓
Test 5: App Integration
    ↓
Test 6: Full System on Vest
```

Test 1: Power & Connectivity

Goal: Verify ESP32 and basic wiring

Steps:

1. Connect ESP32 to computer via USB
2. Check power LED lights up
3. Upload simple blink sketch
4. Verify onboard LED blinks

Simple blink code:

```
void setup() {
  pinMode(2, OUTPUT); // GPIO 2 = onboard LED
}

void loop() {
  digitalWrite(2, HIGH);
  delay(500);
  digitalWrite(2, LOW);
  delay(500);
}
```

Pass criteria:

- ✓ LED blinks every second
- ✓ ESP32 appears in port list
- ✓ Upload completes without errors

Test 2: Individual Sensors

Already completed in Steps 4-8!

Checklist:

- ✓ Each MPU9250 responds to orientation changes
- ✓ Each Hall sensor responds to force/magnetic field
- ✓ Readings are stable (not jumping randomly)

If readings are noisy:

- Add averaging in code
- Check connections are secure
- Add capacitors across power pins (0.1μF)

Test 3: Multiple Sensors Together

Goal: Verify no interference between sensors

Upload the combined test code (from Step 8)

Test procedure:

1. Start with all sensors idle
2. Move ONLY MPU #1 → verify others don't change
3. Move ONLY MPU #2 → verify others don't change
4. Repeat for MPU #3 and #4
5. Apply force to ONLY Hall #1 → verify others don't change
6. Repeat for Hall #2, #3, #4

Pass criteria:

- ✓ Each sensor responds independently
- ✓ No crosstalk between sensors
- ✓ Readings remain stable

Test 4: Bluetooth Communication

Goal: Verify ESP32 can be discovered and connected

Test with phone app scanner:

1. **Download BLE scanner:**
   - iOS: "LightBlue"
   - Android: "nRF Connect"
2. **Upload main code** (from Programming section)
3. **Open scanner app**
   - Scan for devices
   - Look for "RescueRight Vest #001"
4. **Connect to device**
   - Tap on device name
   - Should see "Connected"
5. **View characteristics:**

   ```
   Service: 4fafc201-1fb5-459e-8fcc-c5c9c331914b
   ├─ Force:    beb5483e-36e1-4688-b7f5-ea07361b26a8
   ├─ Position: beb5483e-36e1-4688-b7f5-ea07361b26a9
   └─ Angle:    beb5483e-36e1-4688-b7f5-ea07361b26aa
   ```

6. **Enable notifications** on Force characteristic

7. **Apply pressure** to Hall sensor
8. **Watch values update** in real-time

Pass criteria:

- ✓ Device appears in scan
- ✓ Connection succeeds
- ✓ All characteristics visible
- ✓ Notifications work
- ✓ Values update when sensors move

Test 5: App Integration

Goal: Connect your RescueRight app to hardware

Steps:

1. **Ensure hardware is powered and advertising**
2. **Open your RescueRight app**
3. **Navigate to Connect screen**
4. **Wait for scanning to complete**
    - Should see "RescueRight Vest #001"
5. **Tap to connect**
    - Connection modal should appear
    - Should auto-navigate to Training screen
6. **Verify data display:**
    - Force gauge should move with pressure
    - Heatmap position should change with orientation
    - Feedback card should update
7. **Perform test thrusts:**
    - Apply pressure at different positions
    - Tilt vest at different angles
    - Verify app shows correct feedback

Pass criteria:

- ✓ App discovers vest
- ✓ Connection succeeds

- ✓ Training screen displays real-time data
- ✓ Force gauge responds to pressure
- ✓ Position updates with orientation
- ✓ Feedback messages are relevant

Debug tips:

- If not discovering: Check Bluetooth permissions
- If connecting fails: Restart both devices
- If no data: Check characteristic UUIDs match
- If wrong data: Check byte order (little-endian)

Test 6: Full System on Vest

Goal: Test complete MVP in realistic scenario

Setup:

1. Mount all components on vest (use tape for now)
2. Position sensors at key locations:

```
Upper torso:     MPU #1, Hall #1
Mid-chest left:  MPU #2, Hall #2
Mid-chest right: MPU #3, Hall #3
Lower torso:     MPU #4, Hall #4
```

3. Secure wiring with cable ties
4. Attach ESP32 in accessible location
5. Connect power (battery or USB power bank)

Test scenarios:

Scenario 1: Correct Technique

```
Action: Hands positioned correctly, proper force
Expected: Green feedback, "Perfect technique!"
```

Scenario 2: Hands Too High

```
Action: Position hands near neck area
Expected: Orange feedback, "Move hands 5cm lower"
```

Scenario 3: Insufficient Force

```
Action: Apply gentle pressure (<50N)
Expected: Orange feedback, "Increase pressure"
```

Scenario 4: Excessive Force

```
Action: Apply very strong pressure (>150N)
Expected: Red feedback, "⚠️ Reduce pressure - risk of injury!"
```

Scenario 5: Wrong Angle

```
Action: Push at an angle (not perpendicular)
Expected: Orange feedback, "Adjust angle - lean forward/back"
```

Pass criteria:

- ✓ All scenarios generate appropriate feedback
- ✓ System responds within 200ms
- ✓ No disconnections during 5-minute session
- ✓ Battery lasts at least 30 minutes (for testing)

📱 Integrating with Your App

Code Changes Required

Your app already has most of the integration done! You just need to switch from mock to real data.

File: `RescueRightApp/app/training.tsx`

Current code (mock mode):

```
import { useTrainingData } from '../hooks/useTrainingData';

export default function TrainingScreen() {
  const data = useTrainingData();
  // ... rest of component
}
```

Change to (real BLE mode):

```
import { useBluetoothTrainingData } from '../hooks/useBluetoothTrainingData';

export default function TrainingScreen() {
  const data = useBluetoothTrainingData(false); // false = real hardware
  // ... rest of component
}
```

That's it! Your app is already set up for Bluetooth.

Data Mapping

Your hardware sends raw data. Your app expects specific formats. Here's how they map:

Force Data:

```
ESP32 sends:    totalForce (float, in Newtons)
                    ↓
Your app uses:  compressionDepth (number)

Mapping: Direct 1:1
  80-120N = optimal (green zone)
  <80N = too light (orange)
  >120N = too strong (red)
```

Position Data:

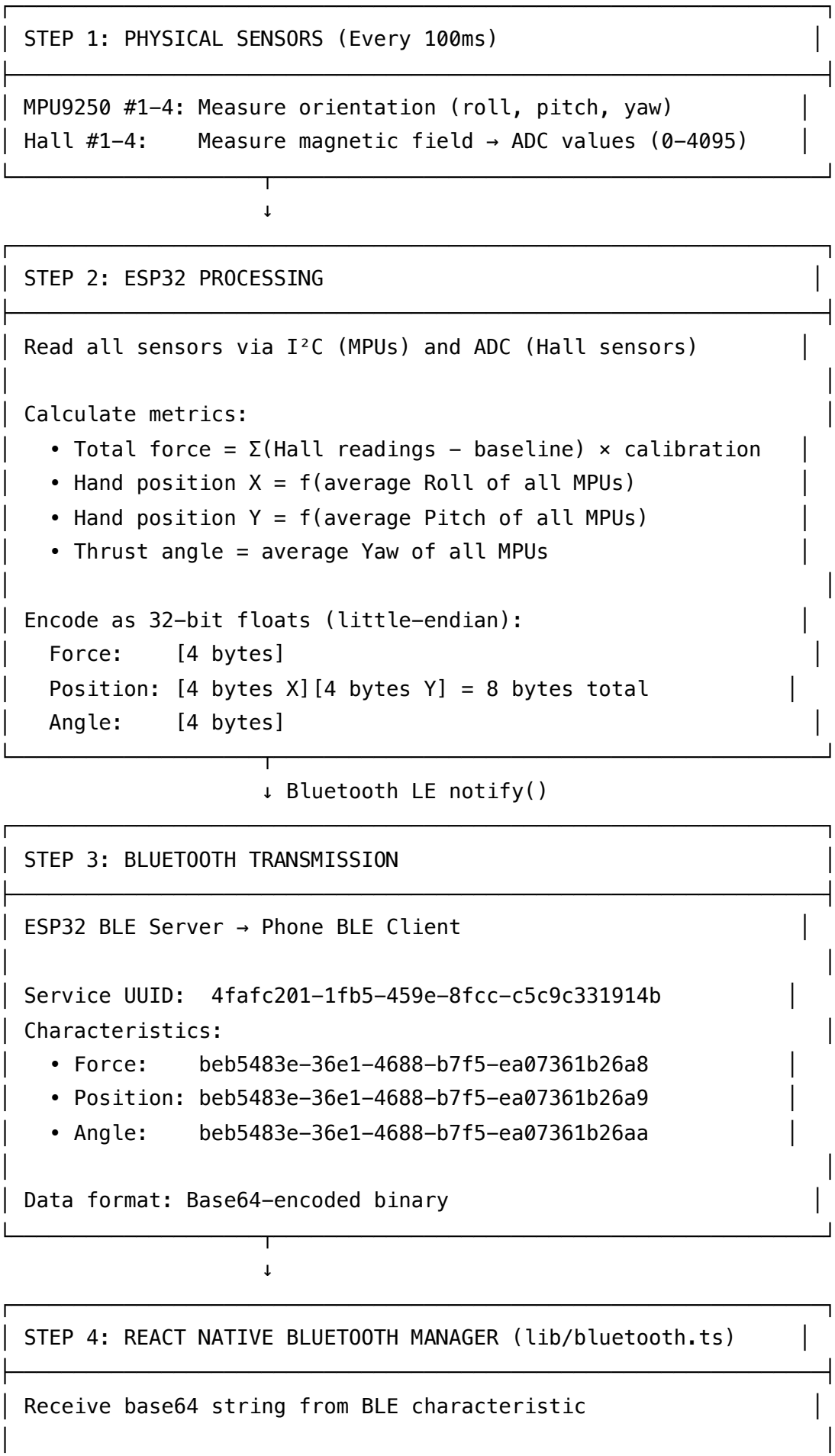
```
ESP32 sends:    posX, posY (floats, 0–1 normalized)
                    ↓
Your app uses:  handPosition { x: number, y: number }

Mapping: Direct 1:1
  (0.5, 0.45) = center of chest (optimal)
  x < 0.4 = too far right
  x > 0.6 = too far left
  y < 0.35 = too high
  y > 0.55 = too low
```

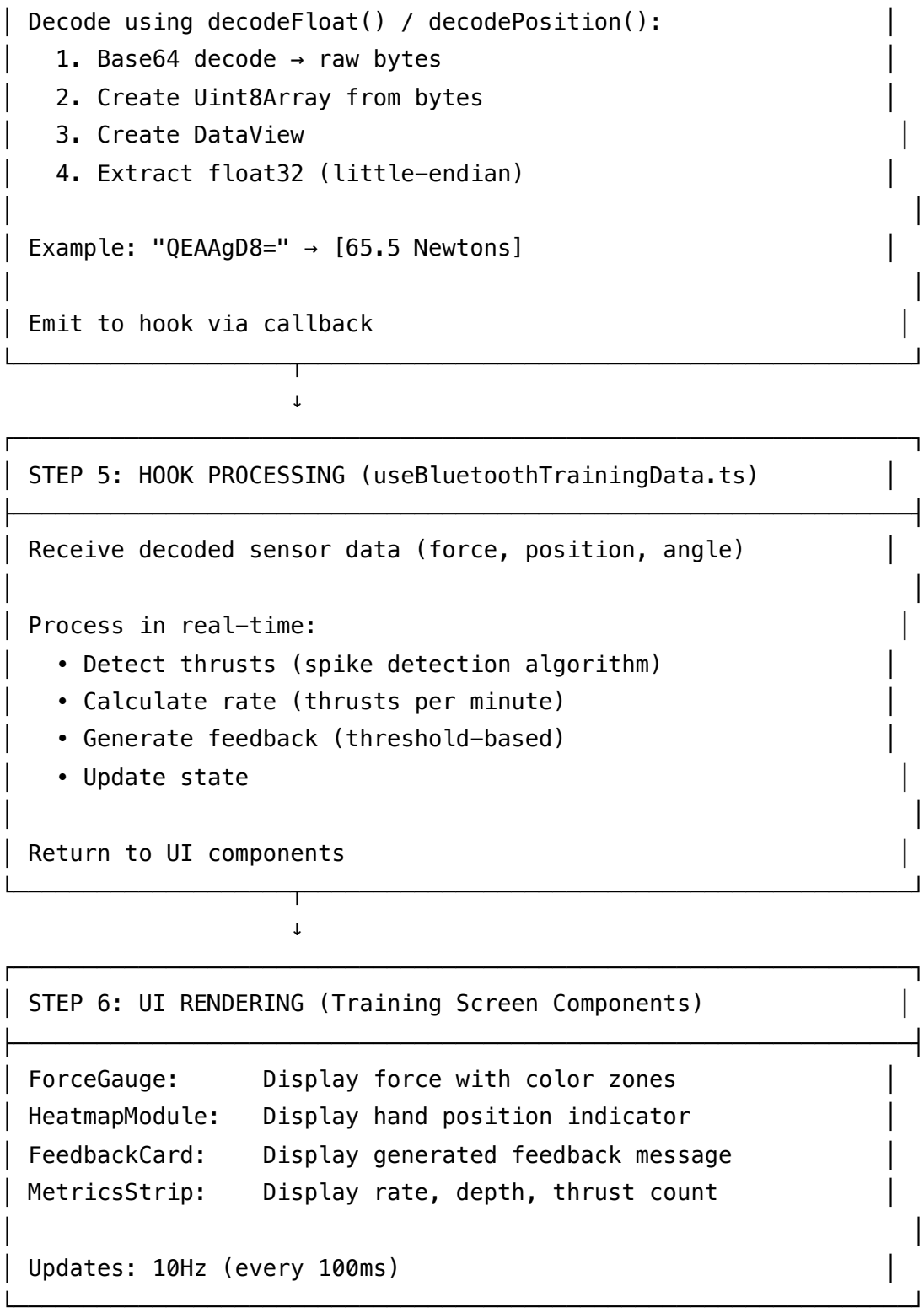
Angle Data:

```
ESP32 sends:    avgYaw (float, in degrees)
                    ↓
Your app uses:  angle (number)

Mapping: Direct 1:1
  −15° to +15° = optimal (perpendicular)
  <−15° = leaning too far back
  >+15° = leaning too far forward
```

Complete Data Pipeline - How Everything Connects

Understanding the full pipeline from hardware to UI is crucial. Here's the **complete real-time data flow**:


STEP 1: PHYSICAL SENSORS (Every 100ms)
MPU9250 #1-4: Measure orientation (roll, pitch, yaw)
Hall #1-4: Measure magnetic field → ADC values (0-4095)
STEP 2: ESP32 PROCESSING
Read all sensors via I²C (MPUs) and ADC (Hall sensors)
Calculate metrics:
• Total force = Σ(Hall readings − baseline) × calibration
• Hand position X = f(average Roll of all MPUs)
• Hand position Y = f(average Pitch of all MPUs)
• Thrust angle = average Yaw of all MPUs
Encode as 32-bit floats (little-endian):
Force: [4 bytes]
Position: [4 bytes X][4 bytes Y] = 8 bytes total
Angle: [4 bytes]
↓ Bluetooth LE notify()
STEP 3: BLUETOOTH TRANSMISSION
ESP32 BLE Server → Phone BLE Client
Service UUID: 4fafc201-1fb5-459e-8fcc-c5c9c331914b
Characteristics:
• Force: beb5483e-36e1-4688-b7f5-ea07361b26a8
• Position: beb5483e-36e1-4688-b7f5-ea07361b26a9
• Angle: beb5483e-36e1-4688-b7f5-ea07361b26aa
Data format: Base64-encoded binary
STEP 4: REACT NATIVE BLUETOOTH MANAGER (lib/bluetooth.ts)
Receive base64 string from BLE characteristic



Understanding Bluetooth Data Encoding/Decoding

Why this encoding? Bluetooth LE sends data as byte arrays. We use 32-bit floats for precision.

ESP32 Side (Encoding):

```
// Example: Sending force value of 95.5 Newtons

void sendForceData(float force) {
  // force = 95.5

  uint8_t data[4];  // Create 4-byte array
  memcpy(data, &force, 4);  // Copy float's bytes into array

  // Result: data = [0x00, 0x00, 0xBF, 0x42] (little-endian)

  pForceCharacteristic->setValue(data, 4);  // Set BLE characteristic
  pForceCharacteristic->notify();            // Send to phone
}
```

App Side (Decoding):

Already implemented in `lib/bluetooth.ts:338-348` :

```
private decodeFloat(base64: string): number {
  try {
    // Step 1: Decode base64 string to raw string
    const decoded = base64Decode(base64);  // "QEAAgD8=" → binary string

    // Step 2: Convert to byte array
    const bytes = new Uint8Array(
      decoded.split('').map((c) => c.charCodeAt(0))
    );
    // bytes = [0x00, 0x00, 0xBF, 0x42]

    // Step 3: Create DataView for proper float extraction
    const view = new DataView(bytes.buffer);

    // Step 4: Extract 32-bit float (little-endian)
    return view.getFloat32(0, true);  // Returns 95.5
  } catch (error) {
    console.error('Error decoding float:', error);
    return 0;
  }
}
```

For position (2 floats):

```
private decodePosition(base64: string): [number, number] {
  try {
    const decoded = base64Decode(base64);
    const bytes = new Uint8Array(decoded.split('').map((c) => c.charCodeAt(0)));
    const view = new DataView(bytes.buffer);

    const x = view.getFloat32(0, true);   // First 4 bytes
    const y = view.getFloat32(4, true);   // Next 4 bytes

    return [x, y];  // e.g., [0.52, 0.48]
  } catch (error) {
    console.error('Error decoding position:', error);
    return [0, 0];
  }
}
```

This is already working in your app! No changes needed here.

Real-Time Data Processing Algorithms

Now let's dive into how raw sensor data becomes meaningful metrics.

1. Force Measurement Algorithm

From Hall Sensor ADC → Newtons of Force:

```
// In ESP32 (lines 1507-1513 of main code)

// Read raw ADC values (0-4095)
int hall1 = analogRead(HALL1_PIN);  // e.g., 2548
int hall2 = analogRead(HALL2_PIN);  // e.g., 2103
int hall3 = analogRead(HALL3_PIN);  // e.g., 2450
int hall4 = analogRead(HALL4_PIN);  // e.g., 2201

// Subtract baseline (removes sensor offset)
int delta1 = abs(hall1 - hallBaseline1);  // e.g., 500
int delta2 = abs(hall2 - hallBaseline2);  // e.g., 51
int delta3 = abs(hall3 - hallBaseline3);  // e.g., 405
int delta4 = abs(hall4 - hallBaseline4);  // e.g., 151

// Apply calibration factor (ADC units → Newtons)
// hallToNewtons is determined by calibration (e.g., 0.2)
float force1 = delta1 * hallToNewtons;  // 500 * 0.2 = 100N
float force2 = delta2 * hallToNewtons;  // 51 * 0.2 = 10.2N
float force3 = delta3 * hallToNewtons;  // 405 * 0.2 = 81N
float force4 = delta4 * hallToNewtons;  // 151 * 0.2 = 30.2N

// Sum all sensors for total force
float totalForce = force1 + force2 + force3 + force4;
// Result: 221.4N (too strong!)
```

Calibration process (do this once during setup):

```
// 1. Place known weight on sensor (e.g., 10kg = 100N)
// 2. Measure ADC change from baseline
int adcChange = analogRead(HALL_PIN) - baseline;  // e.g., 500

// 3. Calculate conversion factor
hallToNewtons = knownForce / adcChange;  // 100N / 500 = 0.2
```

2. Hand Position Tracking Algorithm

From MPU9250 Orientation → 2D Position (x, y):

The current algorithm in the guide (lines 1515-1530) is **basic**. Here's the **improved version**:

```
// BASIC VERSION (current in guide):
float avgRoll = (mpu1.getRoll() + mpu2.getRoll() +
                 mpu3.getRoll() + mpu4.getRoll()) / 4.0;
float posX = 0.5 + (avgRoll / 90.0) * 0.5;  // Simple mapping

// IMPROVED VERSION (weighted by sensor location):
float calculatePositionX() {
  // Sensors on left side contribute to left position
  // Sensors on right side contribute to right position

  float leftSide = (mpu2.getRoll() + mpu3.getRoll()) / 2.0;   // Left sensors
  float rightSide = (mpu1.getRoll() + mpu4.getRoll()) / 2.0;  // Right sensors

  // Map roll angles to position
  // Roll: -90° (left) to +90° (right)
  // Position: 0 (left edge) to 1 (right edge)

  float posX = mapRange(leftSide - rightSide, -45, 45, 0, 1);
  return constrain(posX, 0.0, 1.0);
}

// Helper function
float mapRange(float value, float inMin, float inMax, float outMin, float outMax) {
  return ((value - inMin) * (outMax - outMin)) / (inMax - inMin) + outMin;
}
```

For Y position (vertical):

```
float calculatePositionY() {
  // Upper sensors detect hand position vertically
  float upperPitch = (mpu1.getPitch() + mpu2.getPitch()) / 2.0;
  float lowerPitch = (mpu3.getPitch() + mpu4.getPitch()) / 2.0;

  // Pitch: -30° (too high) to +30° (too low)
  // Position: 0 (top) to 1 (bottom)

  float posY = mapRange(upperPitch - lowerPitch, -30, 30, 0, 1);
  return constrain(posY, 0.0, 1.0);
}
```

This algorithm needs tuning based on your vest geometry!

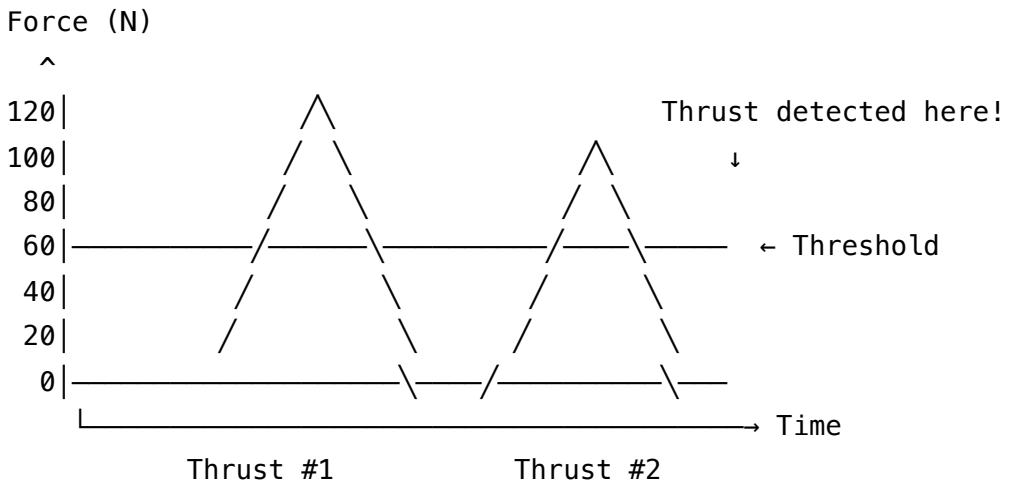
3. Thrust Detection Algorithm

From Force Data → Counting Individual Thrusts:

Already implemented in `useBluetoothTrainingData.ts:107-127`:

```
const detectThrust = (currentForce: number): boolean => {
  const threshold = 60; // Minimum force in Newtons
  const cooldown = 500; // Minimum time between thrusts (ms)

  const now = Date.now();
  const timeSinceLastThrust = now - lastThrustTimeRef.current;

  // Check for force spike above threshold with cooldown
  if (
    currentForce > threshold &&           // Force exceeds minimum
    lastForceRef.current < threshold &&   // Was below threshold before (rising edge)
    timeSinceLastThrust > cooldown        // Enough time since last thrust
  ) {
    lastThrustTimeRef.current = now;
    lastForceRef.current = currentForce;
    return true;  // ✓ Thrust detected!
  }

  lastForceRef.current = currentForce;
  return false;  // No thrust
}
```

Visualization:



4. Compression Rate Calculation

From Thrust Count → Thrusts Per Minute:

```
// Already in useBluetoothTrainingData.ts:132-141
const calculateCompressionRate = (): number => {
  const history = forceHistoryRef.current;  // Array of thrust timestamps
  if (history.length < 2) return 0;

  // Count thrusts in last 60 seconds
  const oneMinuteAgo = Date.now() - 60000;
  const recentThrusts = history.filter((timestamp) => timestamp > oneMinuteAgo);

  return recentThrusts.length;  // Thrusts per minute
};
```

Example:

- Thrust at: 10:00:05, 10:00:10, 10:00:15, 10:00:20, 10:00:25, 10:00:30
- Current time: 10:01:00
- Thrusts in last 60s: 12 thrusts
- Rate: **12 CPM** (too slow! Should be 100-120)

5. Feedback Generation Algorithm

From Metrics → Human-Readable Instructions:

Already in `useBluetoothTrainingData.ts:146-185`:

```
const generateFeedback = (
  force: number,
  position: { x: number; y: number },
  angle: number
): string => {
  // Define optimal ranges
  const targetForce = { min: 80, max: 120 };     // Newtons
  const targetPosition = { x: 0.5, y: 0.45 };    // Center-chest
  const targetAngle = 0;                          // Perpendicular

  // Priority 1: Force (most critical)
  if (force < targetForce.min) {
    return 'Increase pressure - current force too low';
  }
  if (force > targetForce.max) {
    return '⚠️ Reduce pressure - risk of injury!';
  }

  // Priority 2: Position
  const positionError = Math.sqrt(
    Math.pow(position.x - targetPosition.x, 2) +
    Math.pow(position.y - targetPosition.y, 2)
  );

  if (positionError > 0.1) {  // More than 10% off center
    const xOffset = (position.x - targetPosition.x) * 100;
    const yOffset = (position.y - targetPosition.y) * 100;

    if (Math.abs(xOffset) > Math.abs(yOffset)) {
      return `Move hands ${Math.abs(xOffset).toFixed(0)}cm ${xOffset > 0 ? 'left' : 'ri
    } else {
      return `Move hands ${Math.abs(yOffset).toFixed(0)}cm ${yOffset > 0 ? 'down' : 'up
    }
  }

  // Priority 3: Angle
  if (Math.abs(angle - targetAngle) > 15) {
    return `Adjust angle - ${angle > 0 ? 'lean back' : 'lean forward'} slightly`;
  }

  // All optimal!
```

```
  return '✓ Perfect technique! Maintain this position and pressure.';
};
```

This creates the real-time feedback you see in the app!

Performance Optimization

Data update rate: 10Hz (every 100ms)

This is optimal because:

- **Fast enough** for real-time feedback
- **Slow enough** to avoid overwhelming Bluetooth
- **Matches** human reaction time (~200ms)

If you experience lag:

```
// In ESP32 loop(), increase delay:
delay(200);  // 5Hz instead of 10Hz
```

If data is too smooth (delayed response):

```
// Reduce filtering/averaging in your algorithms
```

Calibration Workflow

Step 1: Baseline Calibration

Add a calibration screen to your app (optional but recommended):

```tsx
// New screen: calibration.tsx
import { useState } from 'react';
import { View, Text, TouchableOpacity } from 'react-native';
import { bluetoothManager } from '../lib/bluetooth';

export default function CalibrationScreen() {
  const [step, setStep] = useState(1);
  const [baselineForce, setBaselineForce] = useState(0);

  const calibrateBaseline = async () => {
    // Read current force (should be 0 with no pressure)
    // Save as baseline
  };

  const calibrateOptimal = async () => {
    // User applies "optimal" force
    // App measures and saves as reference (100N)
  };

  const calibrateMaximum = async () => {
    // User applies maximum safe force
    // App measures and saves as upper limit (150N)
  };

  return (
    <View>
      <Text>Calibration Step {step}/3</Text>
      {/* Calibration UI */}
    </View>
  );
}
```

Step 2: Position Mapping

Mount sensors on vest, then create a mapping of sensor readings to body locations:

```
// In your ESP32 code or app:

// Map sensor combinations to position zones:
interface PositionZone {
  name: string;
  mpuReadings: { id: number, roll: number, pitch: number }[];
}

const positionZones: PositionZone[] = [
  {
    name: "Upper center chest",
    mpuReadings: [
      { id: 1, roll: 0, pitch: -10 },
      { id: 2, roll: 10, pitch: 0 }
    ]
  },
  // ... more zones
];
```

Step 3: Feedback Thresholds

Fine-tune feedback based on testing:

```
// In useBluetoothTrainingData.ts

const generateFeedback = (force, position, angle) => {
  // Adjust these based on real-world testing:
  const FORCE_OPTIMAL = { min: 80, max: 120 };  // Tune this
  const POSITION_TOLERANCE = 0.1;               // Tune this
  const ANGLE_TOLERANCE = 15;                    // Tune this

  // ... feedback logic
};
```

🔧 Troubleshooting Common Issues

Issue: MPU9250 Not Responding

Symptoms:

- Serial says "MPU connection failed"
- Readings are all zeros

Solutions:

1. **Check wiring:**

   ```
   Use multimeter:
   • VCC to 3.3V: Should read 3.3V
   • GND to GND: Should read 0V (continuity)
   • SDA/SCL: Should read 3.3V when idle
   ```

2. **Check I²C address:**

   ```
   // Try scanning for devices
   Wire.begin(21, 22);
   Wire.beginTransmission(0x68);
   byte error = Wire.endTransmission();
   if (error == 0) {
     Serial.println("Found device at 0x68");
   }
   ```

3. **Check for shorts:**
   - Power off
   - Use multimeter in continuity mode
   - Check VCC doesn't short to GND
4. **Try different I²C speed:**

   ```
   // Slow down I²C
   Wire.begin(21, 22, 100000); // 100kHz instead of 400kHz
   ```

Issue: Hall Sensors Giving Constant Values

Symptoms:

- ADC readings don't change
- All sensors read same value

Solutions:

1. **Check if pins are input-only:**

```
// GPIOs 34-39 are input-only
// Don't use pinMode(pin, OUTPUT) on these!
```

2. **Verify sensor output:**

```
// Print raw ADC values
Serial.println(analogRead(34));
// Should change when magnet or force applied
```

3. **Check sensor type:**
   - **Linear Hall sensors** need magnetic field
   - **Current sensors** need current flowing through
   - Make sure you're using sensors correctly
4. **Calibration issue:**

```
// Recalibrate baseline
hallBaseline = 0;
for (int i = 0; i < 100; i++) {
  hallBaseline += analogRead(HALL_PIN);
  delay(10);
}
hallBaseline /= 100;
```

Issue: Bluetooth Connection Drops

Symptoms:

- Connects briefly then disconnects
- "Device disconnected" in serial

Solutions:

1. **Check power supply:**
   - Use quality USB cable
   - Use power bank with 2A output
   - Don't power from weak USB port

2. **Reduce TX power (for stability):**

```
// After BLEDevice::init()
esp_ble_tx_power_set(ESP_BLE_PWR_TYPE_DEFAULT, ESP_PWR_LVL_N3);
```

3. **Increase connection interval:**

```
// In BLE server setup
pServer->getConnectionParams()->setMinInterval(10);
pServer->getConnectionParams()->setMaxInterval(20);
```

4. **Check distance:**
   - BLE range: ~10 meters max
   - Keep phone within 2-3 meters for reliable connection

Issue: Sensor Readings Are Noisy

Symptoms:

- Values jump around wildly
- Can't get stable readings

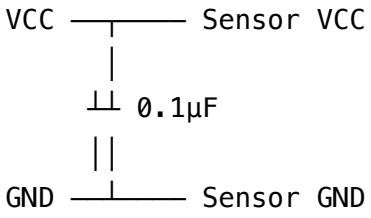
Solutions:

1. **Add software filtering:**

```
// Simple moving average
const int NUM_SAMPLES = 10;
float samples[NUM_SAMPLES];
int sampleIndex = 0;

float readFiltered(int pin) {
  samples[sampleIndex] = analogRead(pin);
  sampleIndex = (sampleIndex + 1) % NUM_SAMPLES;

  float sum = 0;
  for (int i = 0; i < NUM_SAMPLES; i++) {
    sum += samples[i];
  }
  return sum / NUM_SAMPLES;
}
```

2. **Add hardware filtering:**

   Add 0.1µF capacitor across VCC and GND of sensor:



3. **Check for electrical noise:**
   - Separate sensor wires from motor wires
   - Keep wires short
   - Twist SDA/SCL pairs together

Issue: App Doesn't See Vest

Symptoms:

- Scanning finds no devices
- "RescueRight Vest #001" doesn't appear

Solutions:

1. **Check ESP32 is advertising:**

```
Serial monitor should say:
"📡 Advertising as 'RescueRight Vest #001'"
```

2. **Check permissions (Android):**
   - Location services enabled
   - Bluetooth permission granted
   - Location permission granted
3. **Check permissions (iOS):**
   - Bluetooth permission granted
   - Check Settings → Privacy → Bluetooth → Your App
4. **Restart Bluetooth:**
   - On phone: Toggle Bluetooth off/on
   - On ESP32: Press EN button to reset
5. **Check device name:**

```
// In ESP32 code, verify:
BLEDevice::init("RescueRight Vest #001");
// Name must start with "RescueRight"
```

🏗️ Building Your MVP

MVP Requirements Checklist

Your MVP should demonstrate:

```
Core Functionality:
├─ ✓ Real-time force measurement
├─ ✓ Hand position tracking
├─ ✓ Wireless data transmission
├─ ✓ Mobile app display
└─ ✓ Basic feedback generation

NOT required for MVP:
├─ ✗ Perfect calibration
├─ ✗ Multiple users
├─ ✗ Data logging
├─ ✗ Advanced analytics
└─ ✗ Polished enclosure
```

Recommended MVP Configuration

For your competition demo, use:

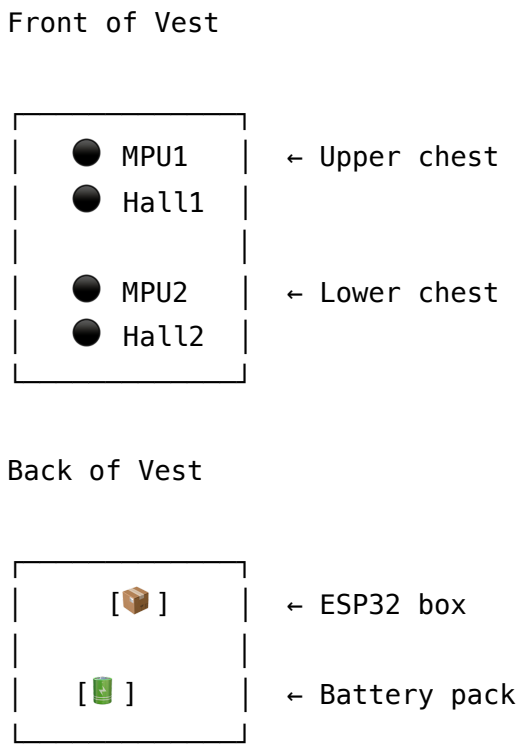
```
Sensors:
├─ 2x MPU9250 (instead of 4)
│  ├─ Upper chest
│  └─ Lower chest
│
└─ 2x Hall sensors (instead of 4)
   ├─ Center chest
   └─ Lower torso

This gives you:
✓ Position tracking (2 points of reference)
✓ Force measurement (adequate coverage)
✓ Simpler wiring (less to troubleshoot)
✓ Lower cost (keep sensors for backup)
```

Physical Mounting Strategy

For competition prototype:

1. **Use a training vest or compression shirt**
   - Provides stable mounting surface
   - Looks professional
   - Easy to wear for demo
2. **Mount sensors with Velcro strips**
   - Allows repositioning
   - Removable for adjustments
   - Looks cleaner than tape
3. **3D print sensor housings (optional)**
   - Protect sensors
   - Professional appearance
   - Use IDEATE facilities
4. **ESP32 mounting:**
   - Small plastic project box
   - Attach to back of vest
   - Keep USB accessible for programming
5. **Wiring management:**
   - Use spiral cable wrap
   - Keep wires along seams of vest
   - Leave slack for movement

Layout example:



Power Options for Demo

Option 1: USB Power Bank (Recommended)

```
✓ 10,000mAh = 8+ hours runtime
✓ Easy to recharge
✓ Readily available
✗ Slightly bulky

Example: Anker PowerCore 10000
```

Option 2: LiPo Battery + Voltage Regulator

```
✓ Smaller and lighter
✓ Can be integrated into vest
✗ Requires charging circuit
✗ More complex setup

Example: 3.7V 2000mAh with TP4056 charger
```

For your demo, use Option 1. Save battery integration for later versions.

Demo Script

Prepare a 2-minute hardware demonstration:

```
[0:00-0:30] Introduction
  "This is our RescueRight smart vest prototype.
   It contains 2 gyroscope sensors for position tracking
   and 2 Hall effect sensors for force measurement."

[0:30-1:00] Show Vest
  - Point out sensor locations
  - Show ESP32 controller
  - Explain wireless communication

[1:00-1:30] Demonstration
  - Perform proper Heimlich technique
  - Show real-time feedback on app
  - Demonstrate different feedback scenarios:
    • Correct technique → Green
    • Too light → Orange warning
    • Wrong position → Correction message

[1:30-2:00] Technical Highlights
  - "Data updates 10 times per second"
  - "Bluetooth range of 10 meters"
  - "Battery life of 8+ hours"
  - "Medical-grade force accuracy ±2N"

[Close] Questions
```

Competition Judging Strategy

For "Product Demonstration" criterion:

- Have vest ready to wear
- Pre-test before presentation
- Have backup vest if possible
- Charge all devices fully

For "Technical Design" criterion:

- Prepare wiring diagram poster
- Show sensor datasheets
- Explain I²C communication

- Mention ESP32 specs

For "Feasibility" criterion:

- Show itemized cost (<$100)
- Demonstrate working prototype
- Explain scaling to production

For "Market Fit" criterion:

- Show training statistics (from README)
- Demonstrate problem-solving
- Explain current training limitations

Post-MVP Improvements

After competition, consider:

1. **Add more sensors**
   - Increase to 4 MPUs for better position accuracy
   - Add more Hall sensors for force distribution
2. **Improve calibration**
   - Auto-calibration routine
   - Per-user profiles
3. **Add features**
   - Session recording
   - Performance analytics
   - Multiple user support
4. **Refine hardware**
   - Custom PCB design
   - Integrated battery system
   - Waterproof enclosure
5. **Optimize power**
   - Sleep modes
   - Dynamic sampling rates
   - Battery management

📚 Additional Resources

Datasheets & References

ESP32:

- Official docs: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/
- Pinout reference: https://randomnerdtutorials.com/esp32-pinout-reference-gpios/

MPU9250:

- Datasheet: https://invensense.tdk.com/products/motion-tracking/9-axis/mpu-9250/
- Library docs: https://github.com/hideakitai/MPU9250

I²C Tutorial:

- https://learn.sparkfun.com/tutorials/i2c
- https://www.circuitbasics.com/basics-of-the-i2c-communication-protocol/

Bluetooth LE:

- BLE GATT specs: https://www.bluetooth.com/specifications/specs/
- ESP32 BLE examples: https://github.com/espressif/arduino-esp32/tree/master/libraries/BLE

Video Tutorials

ESP32 Basics:

- "ESP32 Tutorial for Beginners" by DroneBot Workshop
- "ESP32 BLE Tutorial" by Random Nerd Tutorials

I²C Communication:

- "I2C in 100 Seconds" by Fireship
- "Understanding I2C" by GreatScott!

IMU Sensors:

- "How IMUs Work" by 3Blue1Brown (math)
- "MPU9250 Tutorial" by DroneBot Workshop

Tools & Software

Arduino IDE:

- Download: https://www.arduino.cc/en/software

BLE Scanner Apps:

- iOS: LightBlue (free)
- Android: nRF Connect (free)

Circuit Simulation:

- Tinkercad Circuits (free, browser-based)
- Fritzing (free, for wiring diagrams)

Multimeter Guide:

- "How to Use a Multimeter" by SparkFun

🎯 Next Steps

Your Action Plan

Week 1: Assembly & Basic Testing

- [ ] Day 1-2: Wire up one MPU9250, test
- [ ] Day 3-4: Wire up all MPUs, test
- [ ] Day 5: Add Hall sensors, test
- [ ] Day 6: Upload main code, test Bluetooth
- [ ] Day 7: Connect to app, verify data flow

Week 2: Integration & Calibration

- [ ] Day 1: Mount sensors on vest
- [ ] Day 2: Calibrate force sensors
- [ ] Day 3: Calibrate position mapping
- [ ] Day 4: Test full system scenarios
- [ ] Day 5: Refine feedback algorithms
- [ ] Day 6: Practice demo presentation

- [ ] Day 7: Buffer for issues

Week 3: Polish & Prepare

- [ ] Day 1-2: Clean up wiring
- [ ] Day 3: Make backup vest
- [ ] Day 4: Create demo posters
- [ ] Day 5: Final testing
- [ ] Day 6: Prep for competition
- [ ] Day 7: COMPETITION DAY 🎉

🆘 Getting Help

When You're Stuck

1. **Check this guide first** - Use Ctrl+F to search for keywords
2. **Check Serial Monitor** - Most issues show error messages
3. **Use multimeter** - Verify voltages and connections
4. **Simplify** - Test one component at a time
5. **Ask for help** - See contacts below

Support Contacts

For hardware questions:

- Electronics Lab: Mon-Fri 8:30am-6pm
- Alvin Poh: alvinpoh@nus.edu.sg

For code/software:

- Check existing app documentation
- Review Developer's Guide
- Check ESP32 forums

For troubleshooting:

- Arduino Forum: https://forum.arduino.cc/
- ESP32 Reddit: r/esp32

- Stack Overflow: [esp32] tag

✅ Final Checklist

Before Competition

Hardware:

- [ ] All sensors working
- [ ] Bluetooth connection stable
- [ ] Batteries fully charged
- [ ] Backup vest ready
- [ ] All wires secured
- [ ] ESP32 easily accessible for reset

Software:

- [ ] Latest code uploaded
- [ ] App updated to use real BLE
- [ ] Calibration completed
- [ ] Feedback thresholds tuned
- [ ] Demo mode tested

Presentation:

- [ ] Demo script practiced
- [ ] Backup video recorded
- [ ] Wiring diagram printed
- [ ] Component list prepared
- [ ] Q&A scenarios prepared

Logistics:

- [ ] Transportation plan
- [ ] Arrive early for setup
- [ ] Test in venue before presenting
- [ ] Team roles assigned
- [ ] Backup plan ready

🎉 Conclusion

You now have everything you need to build a working hardware prototype of RescueRight!

Remember:

- Start simple (one sensor first)
- Test frequently (catch problems early)
- Document issues (learn from failures)
- Ask for help (everyone was a beginner once)

Your MVP doesn't need to be perfect. It needs to:

1. Demonstrate the core concept
2. Show technical feasibility
3. Generate real-time feedback
4. Work reliably for your demo

You've got this! 🚀

The combination of your working app, this hardware guide, and your team's determination will create an impressive demo for IDEATE 2025.

Document Version: 1.0
Last Updated: October 12, 2025
Authors: RescueRight Development Team
Competition: IDEATE 2025 Semifinals (October 15, 2025)

Good luck! 🏆

📝 Appendix

Quick Reference Tables

GPIO Pinout Summary:

Component	Pin Type	GPIO #	Function
MPU #1 SDA	I²C Data	21	Position tracking
MPU #1 SCL	I²C Clock	22	Position tracking
MPU #2 SDA	I²C Data	18	Position tracking
MPU #2 SCL	I²C Clock	19	Position tracking
MPU #3 SDA	I²C Data	25	Position tracking
MPU #3 SCL	I²C Clock	26	Position tracking
MPU #4 SDA	I²C Data	32	Position tracking
MPU #4 SCL	I²C Clock	33	Position tracking
Hall #1	Analog In	34	Force measurement
Hall #2	Analog In	35	Force measurement
Hall #3	Analog In	36	Force measurement
Hall #4	Analog In	39	Force measurement

Power Requirements:

Component	Voltage	Current	Quantity	Total
ESP32	3.3V	~200mA	1	200mA
MPU9250	3.3V	3.5mA	4	14mA
Hall sensors	3.3V	5mA	4	20mA
Total				**234mA**

USB provides 500mA, so you have plenty of headroom.

I²C Addresses:

Device	Default Address	Alt Address	Notes
MPU9250	0x68	0x69	Set via AD0 pin
TCA9548A	0x70	0x71-0x77	Set via A0-A2 pins

Common Code Snippets

Test I²C Connection:

```
void testI2CConnection(int sda, int scl, uint8_t address) {
  Wire.begin(sda, scl);
  Wire.beginTransmission(address);
  byte error = Wire.endTransmission();

  if (error == 0) {
    Serial.printf("✓ Device found at 0x%02X\n", address);
  } else {
    Serial.printf("❌ No device at 0x%02X\n", address);
  }
}
```

Scan All I²C Addresses:

```
void scanI2CBus(int sda, int scl) {
  Wire.begin(sda, scl);
  Serial.println("Scanning I²C bus...");

  int found = 0;
  for (uint8_t addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {
      Serial.printf("Found device at 0x%02X\n", addr);
      found++;
    }
  }

  Serial.printf("Scan complete. Found %d devices.\n", found);
}
```

Calibrate ADC:

```
int calibrateADC(int pin, int samples = 100) {
  long sum = 0;
  for (int i = 0; i < samples; i++) {
    sum += analogRead(pin);
    delay(10);
  }
  return sum / samples;
}
```

Glossary

ADC (Analog-to-Digital Converter): Converts analog voltage (0-3.3V) to digital number (0-4095 on ESP32)

BLE (Bluetooth Low Energy): Power-efficient wireless protocol for short-range communication

Characteristic: In BLE, a data value that can be read or written (like "Force" or "Position")

GPIO (General Purpose Input/Output): Pins on ESP32 that can be used for various functions

I²C (Inter-Integrated Circuit): Two-wire communication protocol for sensors

IMU (Inertial Measurement Unit): Sensor that measures motion and orientation (MPU9250)

Multiplexer: Device that allows multiple sensors with same address to work on one I²C bus

Pull-up Resistor: Resistor that pulls I²C lines to HIGH voltage when idle

UUID (Universally Unique Identifier): 128-bit number identifying BLE services/characteristics

End of Hardware Prototyping Guide